

Machine & Deep Learning pour l'Ingénierie des Données

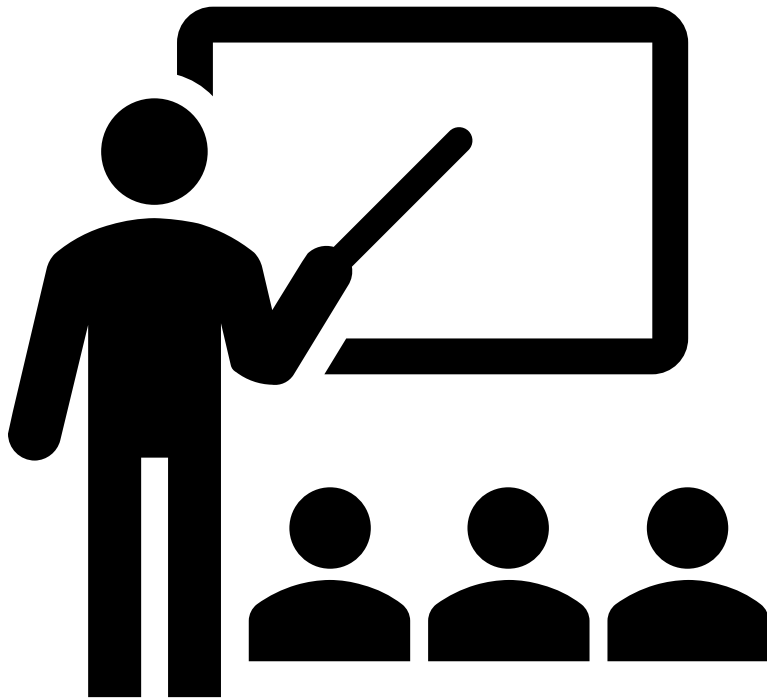
Philip Tchatchoua

2025-2026

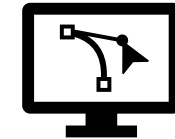


Mastère Data Engineering & IA

Contenu du cours



Fondamentaux du
Machine & Deep
Learning



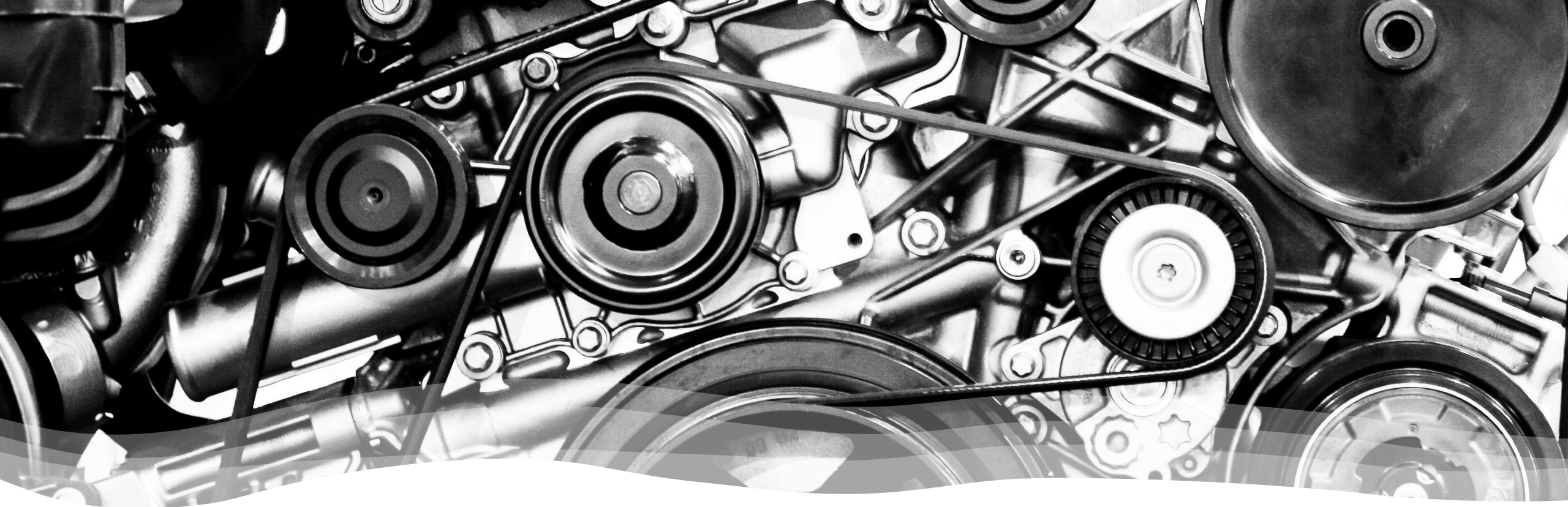
Extraction, préparation,
visualisation et
communication des
données



Modélisation
supervisée et non-
supervisée en Machine
Learning



Modélisation
supervisée et avancée
en Deep Learning



Disposer de données ne suffit pas : il faut apprendre à les transformer en modèles capables de généraliser.

III. Modélisation supervisée et non-supervisée en Machine Learning



Savoir choisir et mettre en œuvre des modèles supervisés et non supervisés adaptés aux données

3.1

Régression : prédire des valeurs continues

3.2

Classification : assigner des classes

3.3

Arbres de décision : apprendre des règles

3.4

Clustering : découvrir des groupes

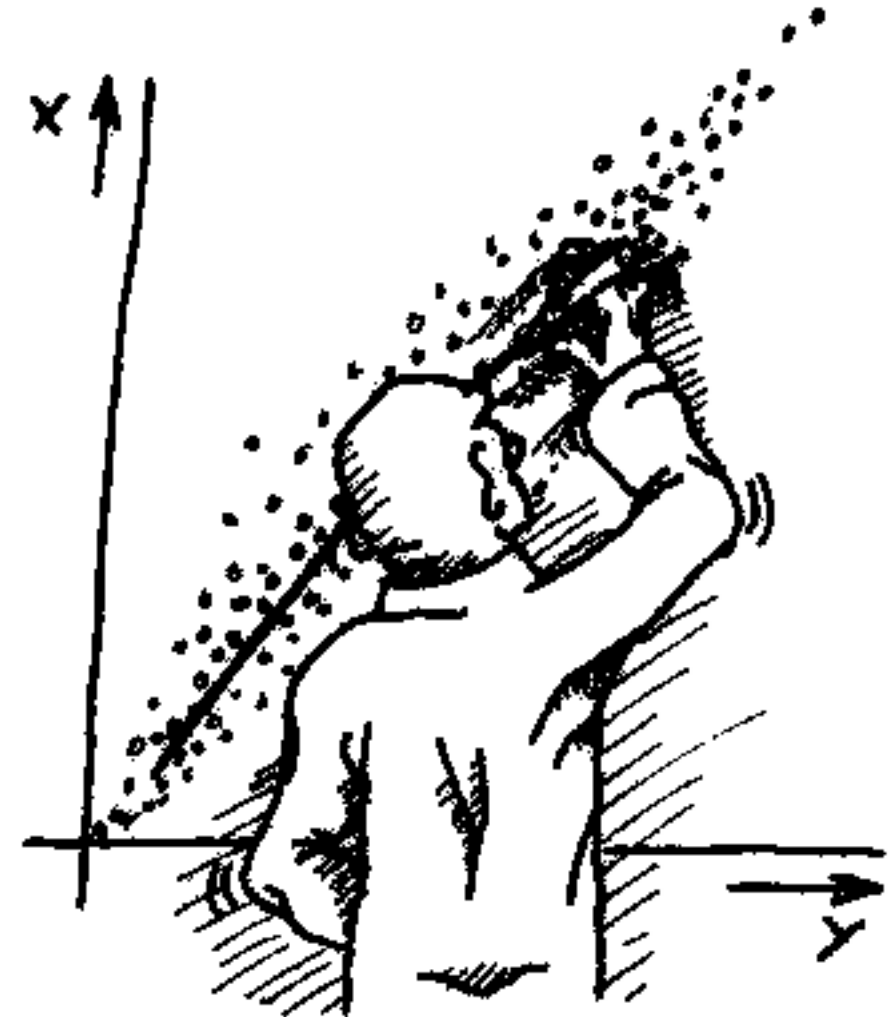
Régression linéaire

La régression linéaire est le **modèle le plus simple**. Il consiste à trouver la meilleure droite qui s'approche le plus des données d'apprentissage. La fonction de prédiction sera donc une droite.

Le modèle prédit par l'algorithme de régression linéaire sera de la forme :

$$y = f(x) = \theta_1 x + \theta_0$$

θ_1 et θ_0 sont les coefficients de la droite.



Régression linéaire

Plus généralement, un modèle linéaire effectue une prédiction en calculant une somme pondérée des variables d'entrée en ajoutant un terme constant.

Le modèle de prédiction d'un algorithme de régression linéaire sera de la forme :

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

y est la valeur prédite,

n est le nombre de variables,

x_i est la valeur de la $i^{\text{ème}}$ variable,

θ_j est le $j^{\text{ème}}$ paramètre du modèle.

On peut l'écrire aussi sous forme vectorielle :

$$y = \theta x$$

θ est le vecteur paramètre du modèle,

x vecteur des valeurs d'une observation,

θx est le produit scalaire de θ et x .

Indicateurs de validation du modèle de prédiction



Comment l'entraîne-t-on ? Et quand arrêter l'entraînement ?

Nous avons **besoin d'une mesure** qui nous dit si le modèle s'ajuste bien (ou mal) aux données d'entraînement.

Root Mean Square Error (RMSE) :

- C'est la racine carrée de l'erreur quadratique moyenne (MSE)
- Soit y_i la valeur réelle et $\hat{y}_i$ la valeur prédite, la fonction coût MSE est donnée par:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

Mean Absolute Error (MAE) :

- C'est l'erreur absolue moyenne qui mesure la moyenne des valeurs absolue des écarts :

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Coefficient de détermination (R^2) :

- Mesure la proportion de la variabilité des données expliquée par le modèle. Il varie entre 0 et 1 : plus il est proche de 1, meilleure est la qualité de l'ajustement du modèle aux données.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Indicateurs de validation du modèle de prédiction



Comment l'entraîne-t-on ? Et quand arrêter l'entraînement ?

- Les deux indicateurs : **MSE** et **R²** sont déjà programmés dans la librairie python : scikit learn
- Tout d'abord, il faudra les importer de la librairie via la commande :

```
from sklearn.metrics import mean_squared_error, r2_score
```

- Nous savons que le RMSE est la racine carrée de MSE. Donc :

```
import numpy as np  
rmse = np.sqrt(mean_squared_error(Y, Y_pred))
```

Le coefficient de détermination R²

```
r2 = r2_score(Y, Y_pred)
```


Entrainement d'un modèle

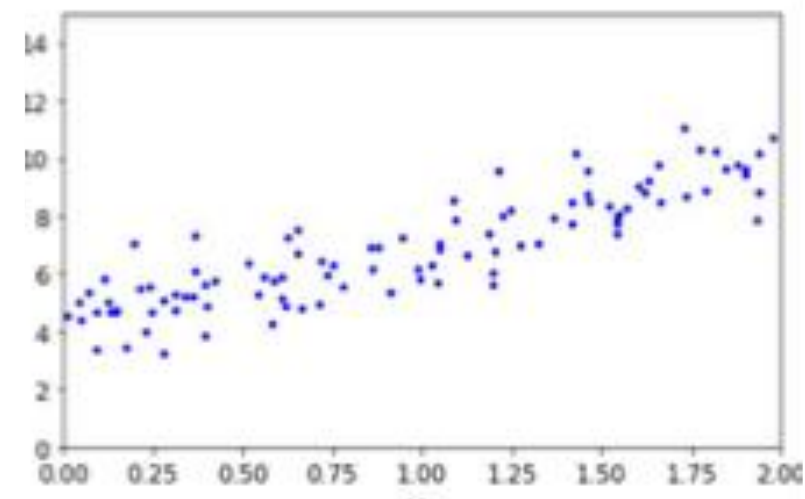
Pour trouver la valeur de $\hat{\theta}$ qui minimise la fonction coût, il existe une solution analytique: une formule mathématique nous fournissant directement le résultat. C'est **l'équation normale**.

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

$\hat{\theta}$ est la valeur de θ qui minimise la fonction coût.

Générons des données à l'allure linéaire sur lesquelles nous allons tester cette équation :

```
import numpy as np
x = 2 * np.random.rand(100, 1)
y = 4 + 3 * x + np.random.randn(100, 1)
```



Entraînement d'un modèle

Pour calculer $\hat{\theta}$ à l'aide de l'équation normal, il faut utiliser `inv()` du module d'algèbre linéaire `np.linalg` de Numpy pour l'inversion de matrice et `dot()` pour les produits matriciels:

```
X = np.c_[np.ones((100, 1)), x] # add x0 = 1 pour chaque instance  
theta_best = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
```

On trouve `theta_best = [[4.1632511 ]`
`[2.90230261]]`

Rappelons qu'on a utilisé l'équation $y = 4 + 3x_0$ pour générer les données.

Nous aurions aimé obtenir $\theta_0 = 4$ et $\theta_1 = 3$, nous avons obtenu : $\theta_0 = 4.16$ et $\theta_1 = 2.9$, ce qui est assez proche mais le bruit n'a pas permis de trouver les paramètres exacts de la fonction d'origine.

Entraînement d'un modèle

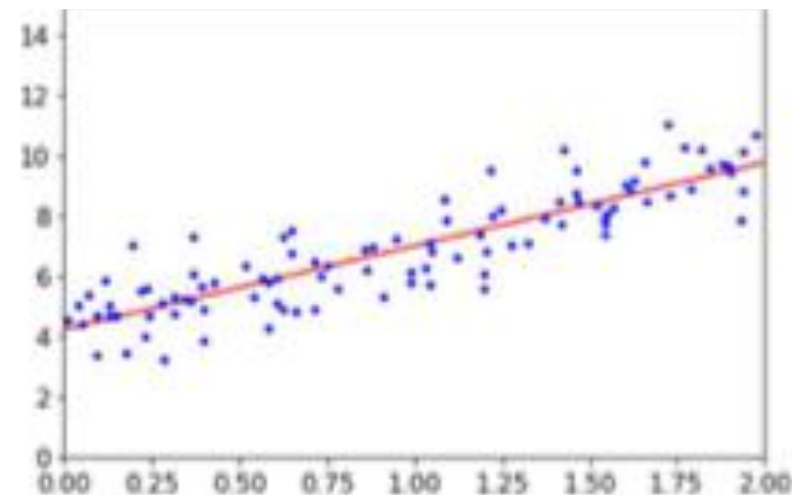
Nous pouvons désormais faire des prédictions à l'aide de $\hat{\theta}$:

```
x_new = np.array([[0], [2]])  
X_new = np.c_[np.ones((2, 1)), x_new] # ajouter x0 = 1 à chaque observation  
y_predict = X_new.dot(theta_best)  
print(y_predict)
```

On aura : $\begin{bmatrix} 4.1632511 \\ 9.96785632 \end{bmatrix}$

On représente graphiquement les prédictions :

```
plt.plot(x_new, y_predict, "r-")  
plt.plot(x, y, "b.")  
plt.axis([0, 2, 0, 15])  
plt.show()
```



Implémentation de la régression linéaire via scikit-learn

Comment effectuer une régression linéaire avec scikit learn:

```
from sklearn.linear_model import LinearRegression  
lin_reg = LinearRegression()  
lin_reg.fit(X, y)  
print(lin_reg.intercept_, lin_reg.coef_)  
print(lin_reg.predict(X_new))
```

On trouve :

```
[[4.07456625]  
 [3.07501075]]
```

 comme coefficients

Et :

```
[[ 4.07456625]  
 [10.22458775]]
```

 comme prédiction

Implémentation de la régression linéaire via scikit-learn

La classe *LinearRegression* repose sur la fonction *scipy.linalg.lstsq()*.

Ce nom signifie « least squares » = modèle des moindres carrés.

On peut également l'appeler directement :

```
X_b = np.c_[np.ones((100, 1)), X] # ajouter x0 = 1 à chaque observation
theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
print(theta_best_svd)
```

On obtient : $\begin{bmatrix} 4.07456625 \\ 3.07501075 \end{bmatrix}$

Cette fonction calcule $\hat{\theta} = X^+y$ où X^+ est le pseudo inverse de X plus précisément le pseudo inverse de Moore-Penrose.

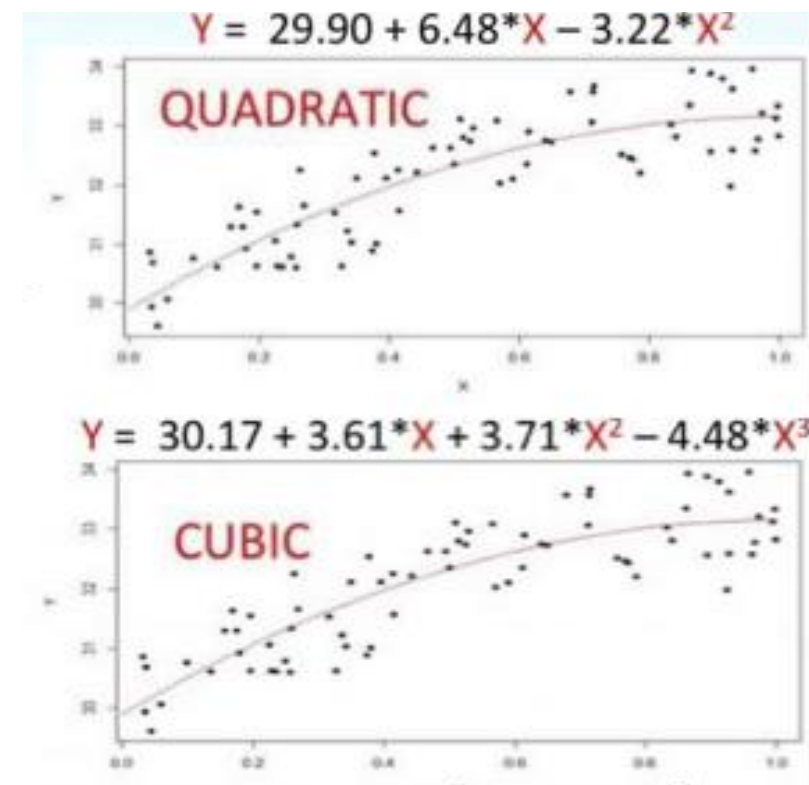
On peut utiliser *np.linalg.pinv()* pour calculer ce pseudo inverse si on le souhaite :

```
print (np.linalg.pinv(X_b).dot(y))
```

On obtient : $\begin{bmatrix} 4.07456625 \\ 3.07501075 \end{bmatrix}$

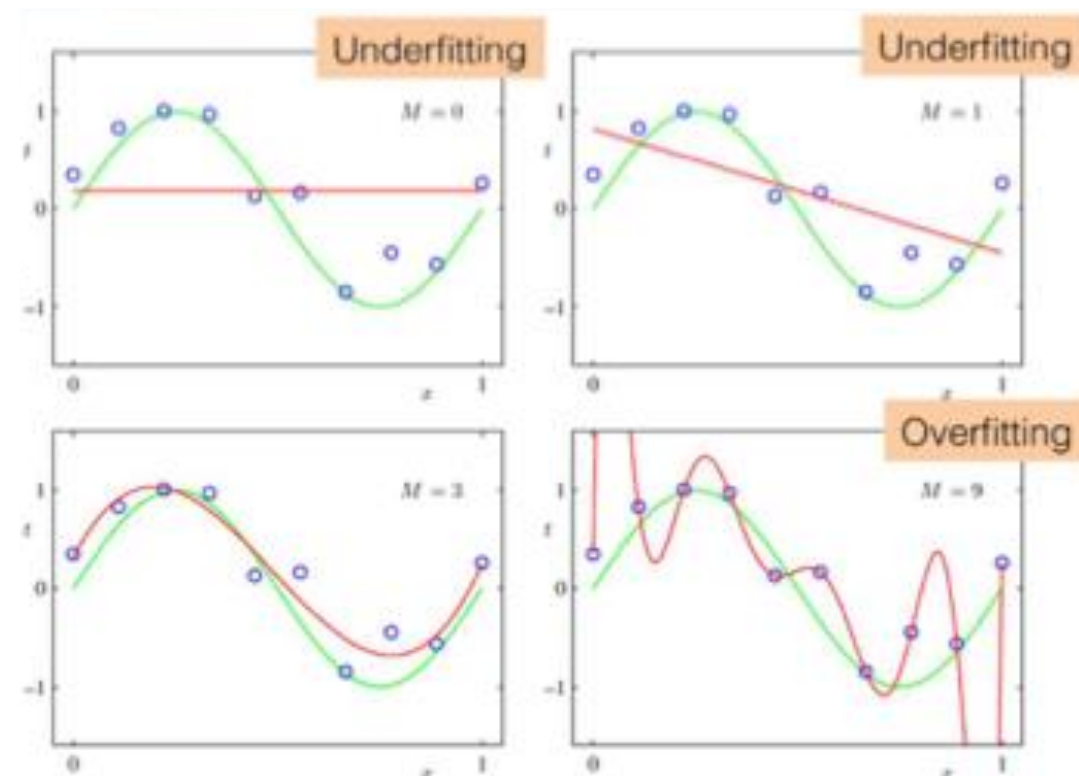
Régression polynomiale

- Pour des données complexes, l'utilisation d'une corrélation linéaire entre les données n'est pas suffisante.
- Les données n'ont pas forcément une relation linéaire entre elles.
- Plusieurs variables prédictives peuvent être nécessaires pour effectuer une prédiction réaliste.
- La **régression polynomiale** et la **régression multivariée** (à plusieurs variables) permettent de calculer des fonctions de mapping complexes qui s'adaptent bien aux données d'apprentissage.



Régression polynomiale

- Pour des données complexes, l'utilisation d'une corrélation linéaire entre les données n'est pas suffisante.
- Les données n'ont pas forcément une relation linéaire entre elles.
- Plusieurs variables prédictives peuvent être nécessaires pour effectuer une prédiction réaliste.
- La **régression polynomiale** et la **régression multivariée** (à plusieurs variables) permettent de calculer des fonctions de mapping complexes qui s'adaptent bien aux données d'apprentissage.
- Attention à ne pas tomber dans le piège du surajustement (Overfitting). Il faut trouver le juste milieu entre le sous-ajustement et le surajustement des données (underfitting et overfitting).



Régression polynomiale

Implémentation de la régression polynomiale

La régression polynomiale est généralement implémentée en utilisant une régression linéaire en faisant certains changement de variables.

Pour cela, l'idée est **d'ajouter les puissances de chacune des variables comme si c'était de nouvelles variables**. Tout d'abord, il faudra les importer de la librairie via la commande :

```
from sklearn.preprocessing import PolynomialFeatures
```

Puis, il faudra appliquer une transformation polynomiale de degré n aux données x pour avoir x_{poly}

```
polynomial_features = PolynomialFeatures(degree=n)  
x_poly = polynomial_features.fit_transform(x)
```


Régression polynomiale

Exemple :

Prenons l'exemple suivant du passage à une courbe quadratique :

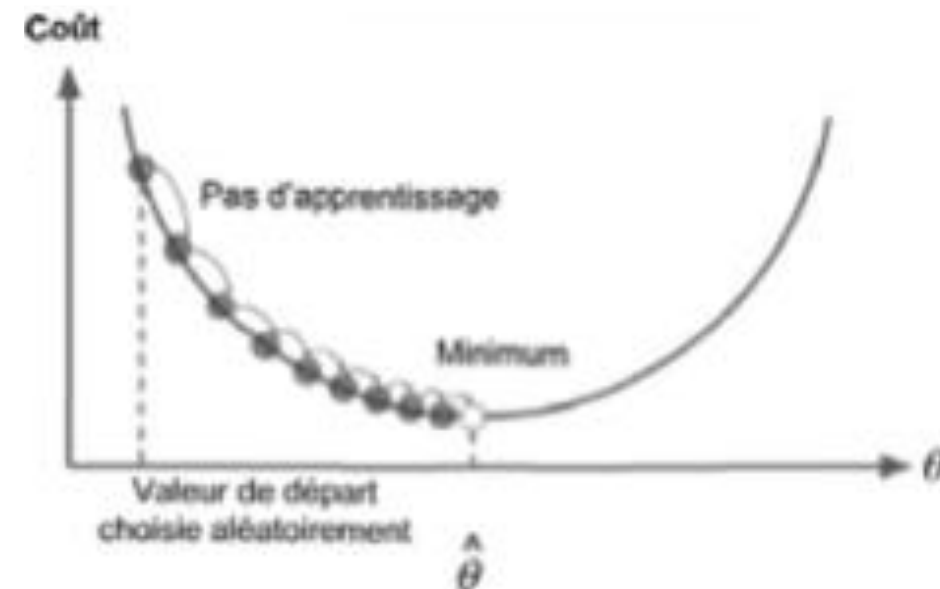
$$X = \begin{bmatrix} -3.29215704 \\ 0.79952837 \\ -0.93621395 \end{bmatrix}$$

Si nous appliquons la transformation polynomiale de degré 2, les vecteurs caractéristiques deviennent :

$$\begin{bmatrix} -3.29215704 & 10.83829796 \\ 0.79952837 & 0.63924562 \\ -0.93621395 & 0.87649656 \end{bmatrix}$$

Descente du gradient

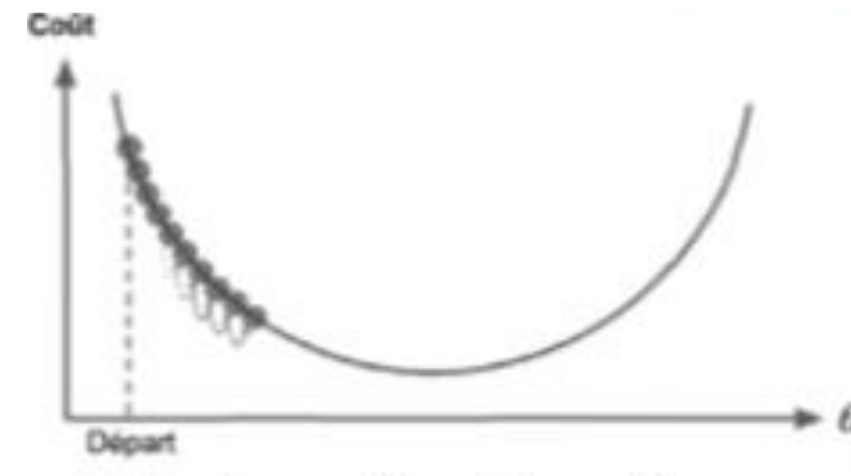
- C'est un algo d'optimisation très général, capable de trouver des solutions optimales à beaucoup de problèmes.
- **L'idée de base** : corriger petit à petit les paramètres dans le but de minimiser une fonction coût.
- **Fonctionnement** : calcule le gradient de la fonction coût au point θ , puis progresse en direction du gradient descendant. Lorsque le gradient est nul on a atteint un minimum.
- On remplit θ avec des valeurs aléatoires, on l'améliore progressivement pas à pas en tentant à chaque étape de faire décroître la fonction coût jusqu'à ce que l'algorithme converge vers un minimum.



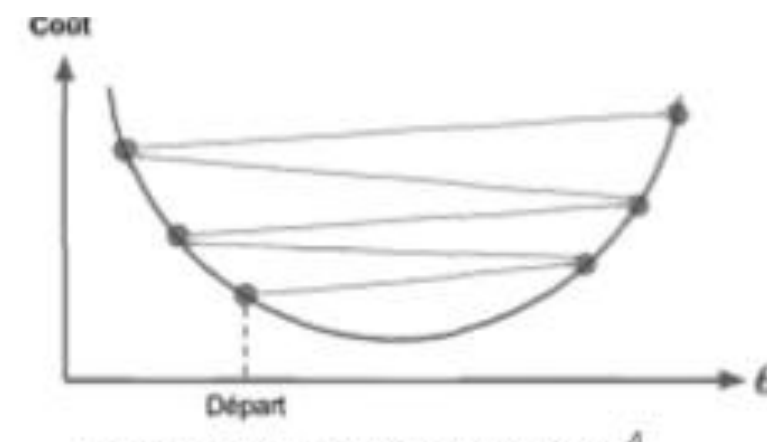
La taille des étapes d'apprentissage est proportionnelle à la pente de la fonction coût : les étapes deviennent + petites à mesure que les paramètres s'approchent du minimum

Descente du gradient

- La dimension du pas est un élément important dans la descente du gradient.
- On le détermine via l'hyperparamètre *learning_rate* (taux d'apprentissage).
- Si le taux d'apprentissage est trop petit: l'algo doit effectuer un grand nombre d'itérations pour converger et prendra beaucoup de temps.
- S'il est trop grand: on risque de dépasser le point le + bas et se retrouver de l'autre côté, peut être même + haut qu'avant => diverger l'algorithme avec des valeurs de + en + grandes => on pourra pas trouver une bonne solution.



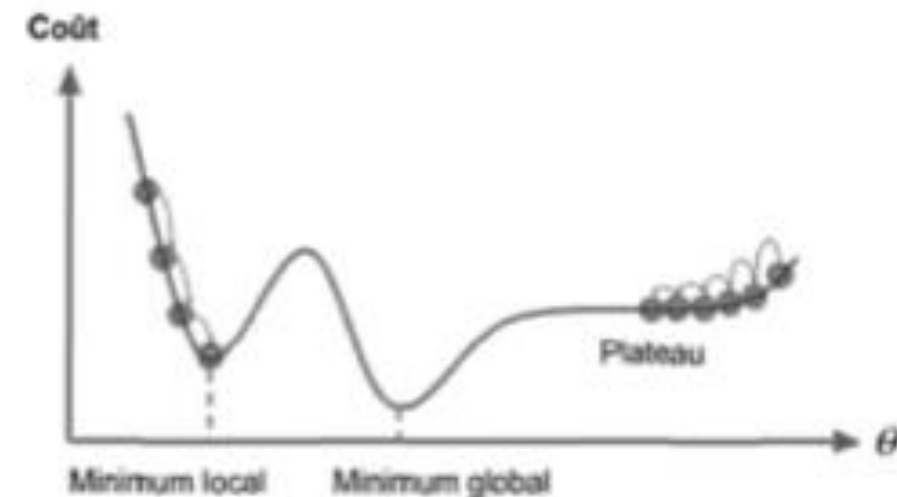
Taux d'apprentissage trop petit



Taux d'apprentissage trop grand

Descente du gradient

- Toutes les fonctions coût n'ont pas la forme d'une jolie cuvette régulière. Il peut y avoir des trous, des crêtes, des plateaux, toutes sortes de terrains irréguliers qui compliquent la convergence vers le minimum.
- Dans l'exemple de la figure suivante :
 - ❑ Si l'initialisation aléatoire démarre l'algo vers la gauche alors il convergera vers un **minimum local** qui n'est pas le **minimum global**.
 - ❑ Si l'initialisation aléatoire démarre l'algo vers la droite, il lui faut très longtemps pour traverser le plateau, si on l'arrête prématurément, on atteindra jamais minimum global.
- Heureusement la fct coût MSE du modèle de régression linéaire est une fonction convexe (donc pas de minimas locaux). C'est aussi une fonction continue dont la pente ne varie jamais abruptement donc on convergera forcément vers le minimum global si on attend assez longtemps.



Pièges de la descente du gradient

Descente du gradient

- La fonction coût a généralement la forme d'un bol. Mais ce bol peut être déformé si les variables ont des échelles très différentes.
- Dans l'exemple de la figure suivante :
 - ❑ A gauche, l'algo descend directement vers le minimum et l'atteint rapidement.
 - ❑ A droite, il part vers une direction presque orthogonal à la direction du minimum global et se termine par une lente progression dans le fond d'une vallée pratiquement plate.
- Il faut que toutes les variables aient la même échelle : on peut utiliser *MinMaxScaler* ou *StandardScaler* de *sklearn*, au risque que la convergence soit bcp plus lente.

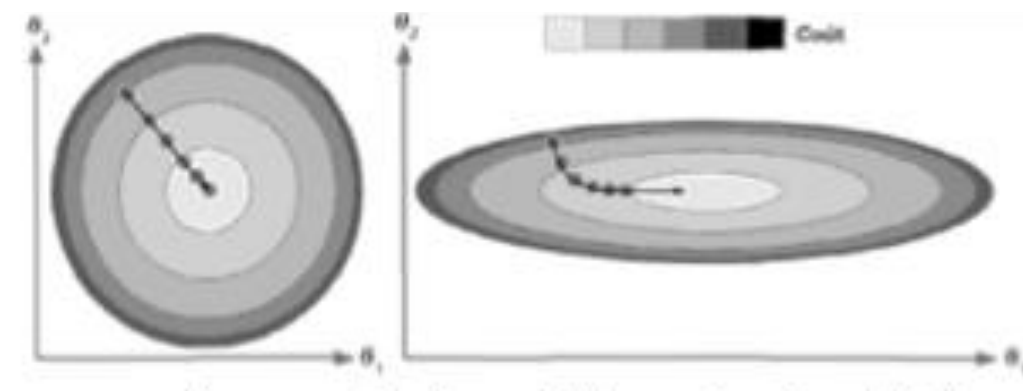


Figure gauche : les variables ont la même échelle.

Figure droite : variable 1 a des valeurs bcp
+petites que la variable 2

Descente du gradient ordinaire

- Pour implémenter il faudra calculer le gradient de la fonction coût par rapport à chaque paramètre θ_j du modèle.
- Quelle est la modification de la fonction coût lorsqu'on modifie un petit peu θ_j : c'est la dérivée partielle.

$$\frac{\partial}{\partial \theta_j} MSE(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)}) x_j^{(i)}$$

- Au lieu de calculer individuellement chaque dérivée partielle, on peut utiliser la formulation vectorielle:
- Ce qui implique des calculs sur tout le jeu d'entraînement à chaque étape de la descente du gradient => très lent sur des jeux d'entraînement volumineux.
- La descente du gradient s'accommode bien à un grand nombre de variables.
- Lorsqu'il y a des centaines de milliers de variables, un modèle de régression linéaire entraîné avec une descente de gradient est bien plus rapide que celui entraîné avec une équation normale.

$$\nabla_{\theta} MSE(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\theta) \\ \frac{\partial}{\partial \theta_1} MSE(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} MSE(\theta) \end{pmatrix} = \frac{2}{m} X^T (X\theta - y)$$

Descente du gradient ordinaire

- Le pas de descente de gradient est donné par (avec η le taux d'apprentissage) : $\theta^{(\text{étape suivante})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$
- Si on reprend l'exemple du dernier cours pour le problème de régression linéaire :

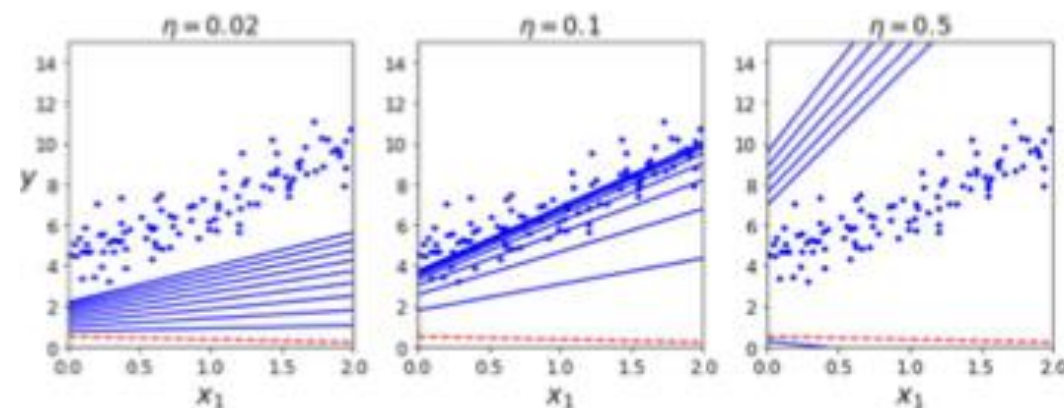
```
eta = 0.1 # taux d'apprentissage
n_iterations = 1000
m=100
theta = np.random.randn(2,1) # initialisation aléatoire
for iteration in range(n_iterations):
    gradients = 2/m * X.T.dot(X.dot(theta) - y)
    theta = theta - eta * gradients
print(theta)
print(X_new.dot(theta))
```

Theta = ([[4.21509616],
[2.77011339]])
et le résultat : ([[4.21509616],
[9.75532293]])

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} X^T (X\theta - y)$$

Descente du gradient ordinaire

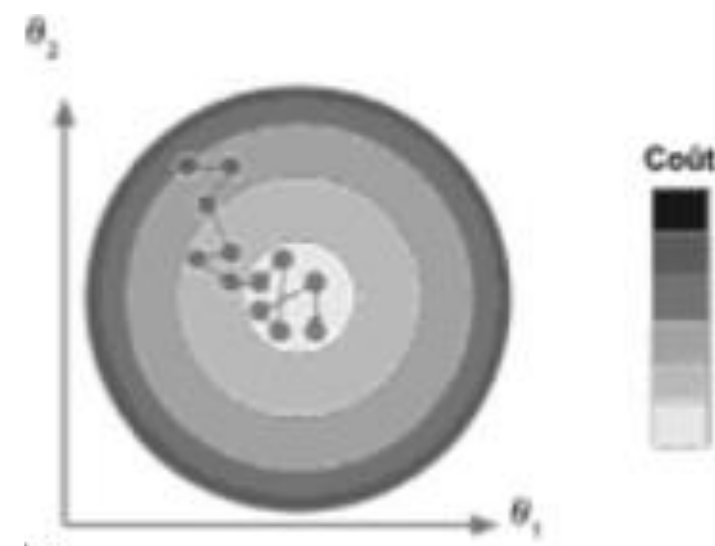
- Que serait-il passé avec un taux d'apprentissage différent ?
- Dans l'exemple de la figure suivante :
- A gauche, taux d'apprentissage trop faible, l'algorithme aboutira mais ça prendra très longtemps.
- Au milieu, taux d'apprentissage semble bon, en quelques itérations l'algorithme a convergé.
- A droite, taux d'apprentissage trop haut, l'algorithme diverge, sautant ici et là et s'éloignant de la solution.
- Généralement on choisit un nombre très grand d'itérations. Mais on interrompt l'algorithme quand le vecteur gradient devient très petit. C'est à dire quand sa norme devient inférieure à un très petit nombre qu'on appelle tolérance.



Descente du gradient avec différents taux d'apprentissage

Descente du gradient stochastique

- Le principal inconvénient de la descente de gradient classique c'est qu'elle utilise à chaque étape l'ensemble du jeu d'entraînement, ceux qui la rend très lente lorsque ce jeu est de grande taille.
- La descente de gradient stochastique choisit à chaque étape une observation prise au hasard dans le jeu d'entraînement et calcule les gradients.
- **Avantages** : Travailler sur une seule observation à la fois rend l'algorithme plus rapide. Il n'a besoin que d'une seule observation en mémoire à chaque itération (implémentation hors mémoire).
- Lorsque la fct coût est très irrégulière, il a + de chances de trouver un minimum global puisqu'il peut sauter à l'extérieur d'un minimum local.
- **Inconvénients** : étant donné sa nature aléatoire, il est bcp moins régulier qu'une descente de gradient ordinaire.
- Au lieu de décroître doucement jusqu'à atteindre le min, la fonction coût va effectuer des sauts vers le haut et vers le bas et ne décroîtra qu'en moyenne.
- Lorsque l'algorithme s'arrête, les valeurs finales des paramètres sont **bonnes mais pas optimales**.



Descente du gradient
stochastique

Descente du gradient stochastique

Problème : une sélection aléatoire non satisfaisante car l'algorithme ne s'arrêtera jamais au minimum.

Solution : réduire progressivement le taux d'apprentissage.

- Les pas sont grands au début (progresser rapidement et échapper aux minimas locaux).
- Puis, ces pas réduisent progressivement pour s'arrêter au minimum global.
- Etant donné que les observations sont choisies au hasard, certaines peuvent être sélectionnées plusieurs fois, alors que d'autres jamais. Si on veut être sûr que l'algorithme parcourt toutes les observations à chaque epoch, on mélange généralement le jeu d'entraînement comme on bat un jeu de cartes.
- La fonction qui détermine ce taux d'apprentissage à chaque étape est appelée l'échéancier d'apprentissage (*learning schedule* en anglais).

Descente du gradient stochastique

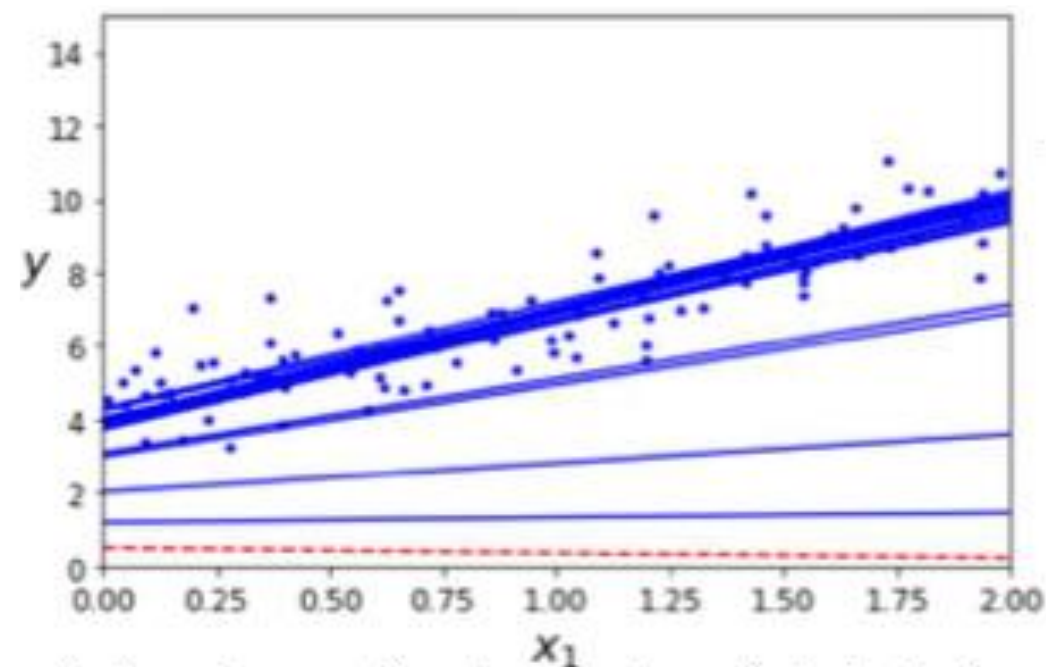
```
n_epochs = 50  
t0, t1 = 5, 50 # hyperparameter d'échéancier d'apprentissage
```

```
def learning_schedule(t):  
    return t0/(t+t1)
```

```
theta = np.random.randn(2,1) # initialisation aléatoire
```

```
for epoch in range(n_epochs):  
    for i in range(m):  
        random_index = np.random.randint(m)  
        xi = X_b[random_index:random_index+1]  
        yi = y[random_index:random_index+1]  
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)  
        eta = learning_schedule(epoch * m + i)  
        theta = theta - eta * gradients
```

```
theta = [[4.21076011],  
         [2.74856079]]
```



Les vingt premiers pas d'une descente de gradient stochastique

Descente du gradient stochastique

Implémentation avec Scikit Learn, on utilise la classe *SGDRegressor*

```
from sklearn.linear_model import SGDRegressor  
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1, random_state=42)  
sgd_reg.fit(X, y.ravel())  
print(sgd_reg.intercept_, sgd_reg.coef_)
```

Affichage : array([4.24365286]), array([2.8250878])

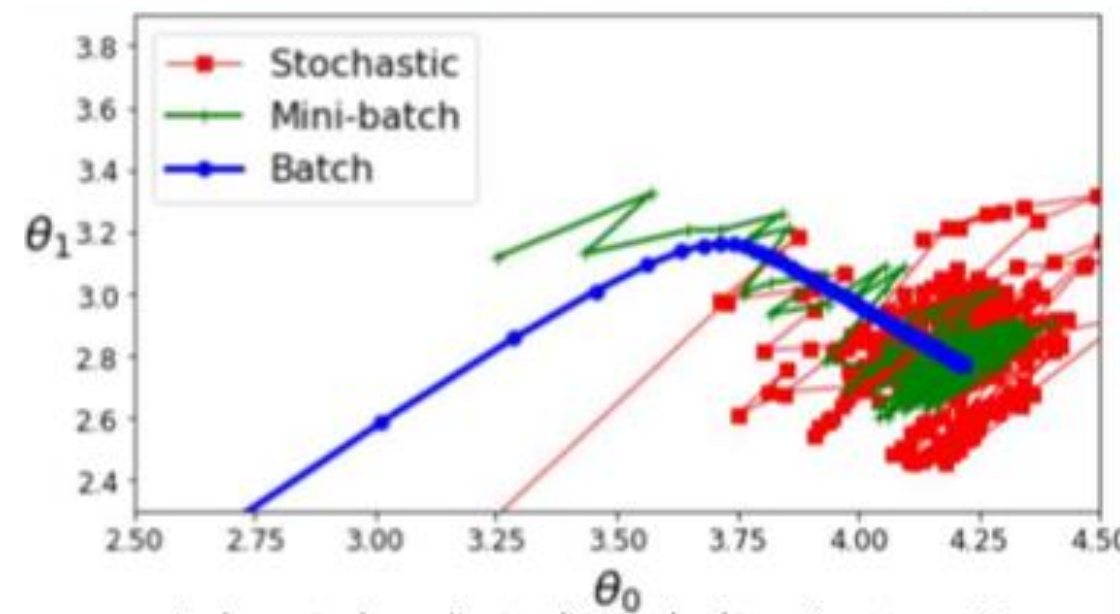
Le code effectue 1000 cycles ou epoch ou tourne jusqu'à ce que la perte devienne inférieure à 0,001 durant une epoch.

Il commence à un taux 0,1 en utilisant le calendrier d'apprentissage par défaut.

A la fin il n'effectue aucune régularisation (penalty=None).

Descente du gradient par mini-lots

- **Principe de base** : Il calcule le gradient sur de petits sous ensembles d'observations sélectionnés aléatoirement qu'on appelle mini-lots (mini-batch en anglais)
- **Avantage** : La progression de l'algorithme dans l'espace des paramètres est moins désordonnée que le gradient stochastique.
- Les pas sont grands au début (progresser rapidement et échapper aux minimas locaux)
- Ça aboutira un peu plus près du minimum comparé à une descente de gradient stochastique.
- Par contre, on aura du mal à sortir d'un minimum local.



La descente de gradient ordinaire s'arrête vraiment au minimum. Tandis que le stochastique ou le mini-lots continue à se déplacer autours.

Descente du gradient par mini-lots

```
n_iterations = 50
minibatch_size = 20
np.random.seed(42)
theta = np.random.randn(2,1) # initialisation aléatoire t0, t1 = 200, 1000
def learning_schedule(t):
    return 1/(t+1)
t=0
for epoch in range(n_iterations):
    shuffled_indices = np.random.permutation(m)
    X_b_shuffled = X_b[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for i in range(0, m, minibatch_size): t+=1
        xi = X_b_shuffled[i:i+minibatch_size]
        yi = y_shuffled[i:i+minibatch_size]
        gradients = 2/minibatch_size * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(t)
        theta = theta - eta * gradients
print(theta)
```

Affichage:
theta = [[4.25214635],
[2.7896408]]

Descente du gradient

Comparaison des algorithmes de regression linéaires

Algorithme	m grand	Hors-mémoire possible	n grand	Hyperparamètres	Normalisation requise ?	Scikit Learn
Équation normale	rapide	non	lent	0	non	Non disponible
SVD	rapide	non	lent	0	non	<i>LinearRegression</i>
DG ordinaire	lent	non	rapide	2	oui	<i>SGDRegressor</i>
DG stochastique	rapide	oui	rapide	≥ 2	oui	<i>SGDRegressor</i>
DG par mini-lots	rapide	oui	rapide	≥ 2	oui	<i>SGDRegressor</i>

Régression logistique

Objectif

- Prédire une variable cible binaire (oui/non, succès/échec, 0/1) en fonction d'un ou plusieurs prédicteurs.

Différence avec la régression linéaire

- Contrairement à la régression linéaire, qui suppose une relation linéaire entre les variables indépendantes et la variable dépendante, la régression logistique applique une transformation sigmoïde aux prédictions pour garantir qu'elles restent comprises entre 0 et 1.

Applications de la régression logistique

- Médecine : Prédire la probabilité qu'un patient ait une maladie en fonction de ses symptômes.
- Finance : Estimer la probabilité de défaut de paiement d'un emprunteur.
- Marketing : Déterminer si un client va acheter un produit en fonction de son comportement.
- Réseaux Sociaux : Déterminer si un message est un spam ou non

Régression logistique

Modèle mathématique : fonction sigmoïde

La régression logistique repose sur la fonction sigmoïde (logistique) définie par :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

où z est une combinaison linéaire des variables explicatives :

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \dots + \beta_n X_n$$

La sortie $\sigma(z)$ correspond à une probabilité $P(Y = 1|X)$

- Si $\sigma(z) \approx 1$, alors $y = 1$ (ex: un patient est malade)
- Si $\sigma(z) \approx 0$, alors $y = 0$ (ex: un patient est sain)
- Si $\sigma(z) \approx 0.5$, la décision est incertaine

Ainsi, la **régression logistique permet de classer une observation en 0 ou en 1 en fonction d'un seuil** (généralement 0.5)

Régression logistique

Modèle mathématique : formulation probabiliste et fonction de vraisemblance

La probabilité que la variable Y prenne la valeur 1 est donnée par :

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \dots + \beta_n X_n)}}$$

Et la probabilité que la variable Y prenne la valeur 0 est :

$$P(Y = 0|X) = 1 - P(Y = 1|X)$$

Étant donné un ensemble de m observations $\{(x_i, y_i)\}_{i=1}^m$, la vraisemblance du modèle est :

$$L(\beta) = \prod_{i=1}^m P(Y_i = 1|X_i)^{y_i} P(Y_i = 0|X_i)^{(1-y_i)}$$

On maximise le **log-vraisemblance** pour estimer les coefficients β :

$$\log L(\beta) = \sum_{i=1}^m [y_i \log P(Y_i = 1|X_i) + (1 - y_i) \log P(Y_i = 0|X_i)]$$

Régression logistique

Apprentissage des paramètres : Estimation des coefficients

La régression logistique utilise **la maximisation de la vraisemblance** pour estimer les coefficients β . Puisque l'équation de vraisemblance est non linéaire, on utilise **la descente de gradient** :

$$\beta_{j+1} = \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$$

Où la fonction de coût (erreur logistique) est :

$$J(\beta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_\beta(X_i)) + (1 - y_i) \log(1 - h_\beta(X_i))]$$

Avec $h_\beta(X_i) = P(Y_i = 1|X_i)$.

Où $h_\beta(X_i)$ est la probabilité prédite pour l'observation i .

Régression logistique

Apprentissage des paramètres : Algorithme de descente du gradient

Quelles sont les valeurs optimales de β ?

1. Initialiser β à des valeurs aléatoires.
2. Calculer les predictions $h_{\beta}(X)$.
3. Mettre à jour les paramètres :

$$\beta_{j+1} = \beta_j - \alpha \sum_{i=1}^m (h_{\beta}(X_i) - y_i) X_{ij}$$

4. Répéter jusqu'à la convergence.

Une fois que les probabilités prédites obtenues, on choisit un seuil τ pour classifier les observations :

$$\hat{y} = \begin{cases} 1, & \text{si } P(Y = 1|X) \geq \tau \\ 0, & \text{sinon} \end{cases}$$

Un seuil courant est **0.5**, mais il peut être ajusté en fonction du problème.

Régression logistique

Exemple d'implémentation Python

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Génération de données aléatoires
np.random.seed(0)
X = np.random.rand(100, 2)
y = (X[:, 0] + X[:, 1] > 1).astype(int)

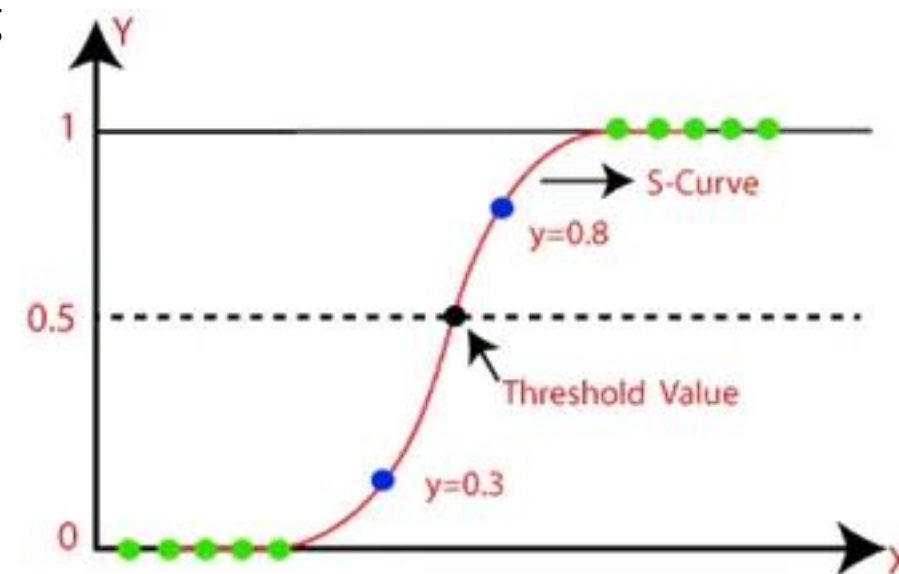
# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entraînement du modèle
model = LogisticRegression()
model.fit(X_train, y_train)

# Prédiction et évaluation
y_pred = model.predict(X_test)
print("Précision :", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Régression logistique

- La régression logistique est un modèle puissant pour la classification binaire.
- La régression logistique est un algorithme de machine learning supervisé utilisé pour résoudre des problèmes de **classification binaire** (ex: spam / non spam, malade / non malade).
- Contrairement à la régression linéaire, qui prédit des valeurs continues, la régression logistique prédit **des probabilités** associées à deux classes (0 ou 1).
- Elle repose sur une approche probabiliste et est largement utilisée en machine learning et en statistiques.
- L'apprentissage se fait par **maximisation de la vraisemblance** et **descente de gradient**.
- L'optimisation des seuils de décision et l'analyse des coefficients sont essentielles pour interpréter les résultats et améliorer les performances du modèle.



III. Modélisation supervisée et non-supervisée en Machine Learning



Savoir choisir et mettre en œuvre des modèles supervisés et non supervisés adaptés aux données

3.1

Régression : prédire des valeurs continues

3.2

Classification : assigner des classes

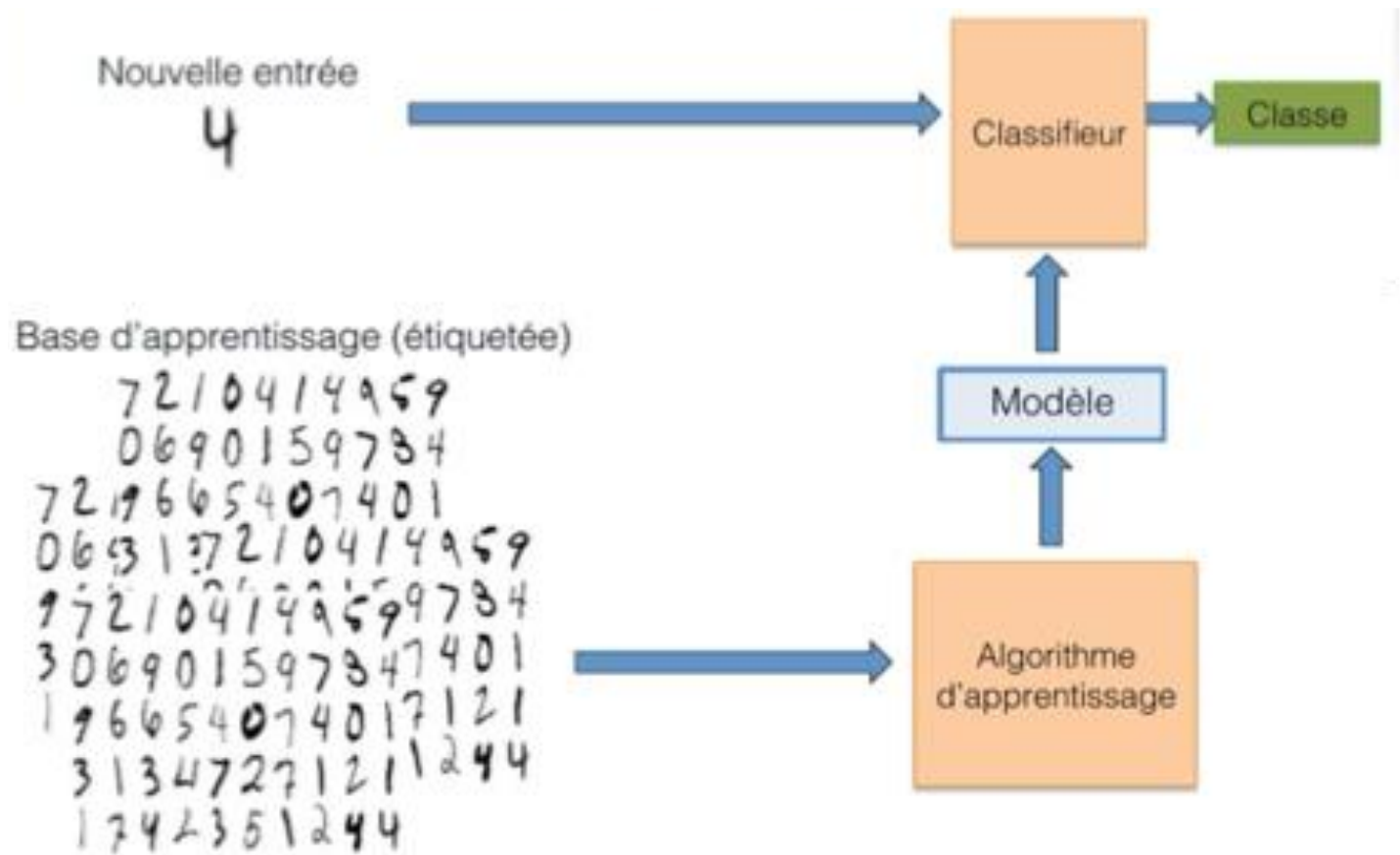
3.3

Arbres de décision : apprendre des règles

3.4

Clustering : découvrir des groupes

Prédire des catégories



La base de données Iris

- Nous allons utiliser la base de données d'Iris de la librairie scikit-learn.
- Cette base contient 150 fleurs d'iris de 3 espèces différentes : Iris setosa, Iris versicolor et Iris virginica.
- On a des caractéristiques pour chaque fleur d'iris: les longueurs et largeurs des sépales et pétales.
- **But** : Déterminer automatiquement l'espèce d'une nouvelle plante en fonction de la mesure des dimensions de ses sépales et pétales que nous aurions réalisée.



Iris Versicolor

Iris Setosa

Iris Virginica

Support Vector Machines (SVM)

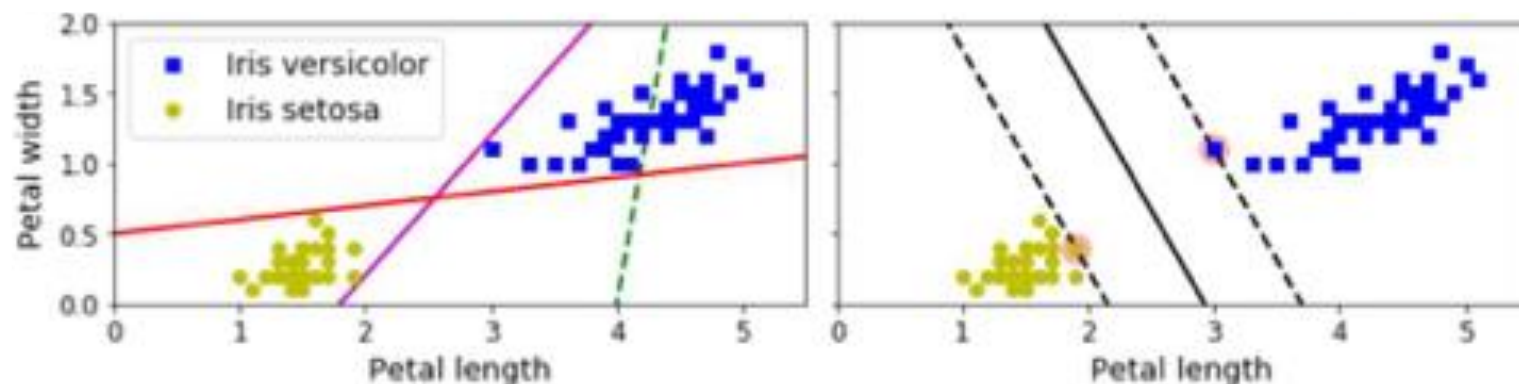
- Les **Support Vector Machines** sont appelés en français : **Séparateur à Vaste Marge** (SVM).
- Ils font partie des algorithmes d'apprentissage automatique initialement définis pour la discrimination (la prévision d'une variable qualitative binaire). Puis, ils ont été généralisés à la prévision d'une variable quantitative.
- Ils sont largement utilisés dans de nombreux types d'applications :
 - Reconnaissance de formes,
 - Reconnaissance de séquences génomiques,
 - Reconnaissance de caractères,
 - détection de spams,
 - Aide au diagnostic médical
 - ...
- Ils représentent l'un des modèles les plus prisés en apprentissage.
- Ils sont un concurrent sérieux des algorithmes les plus performants.

Classification SVM linéaire

SVM : ajuste le chemin le + large possible entre les classes = une classification à large marge

Les 2 classes sont **linéairement séparables**.

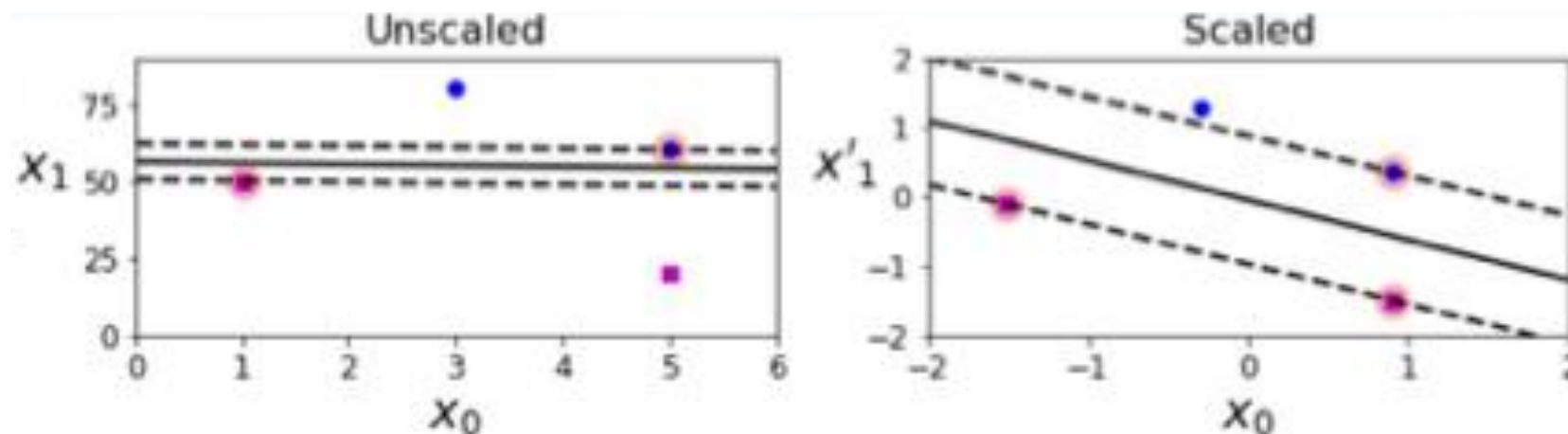
A gauche : frontières de décision de 3 classifieurs linéaires. A droite : frontière de décision d'un SVM.



- **A gauche** : Le classificateur représenté par des tirets est si mauvais qu'il ne sépare pas les 2 classes. Les 2 autres fonctionnent bien sur le jeu d'entraînement, mais les frontières sont si proches des observations qu'ils ne donneront probablement pas de bons résultats sur une nouvelle observation de test.
- **A droite** : La ligne ne se contente pas seulement de séparer les 2 classes mais elle reste aussi distante que possible des observations les plus proches. Les observations au bord du chemin sont appelées **vecteurs de support**.

Classification SVM linéaire

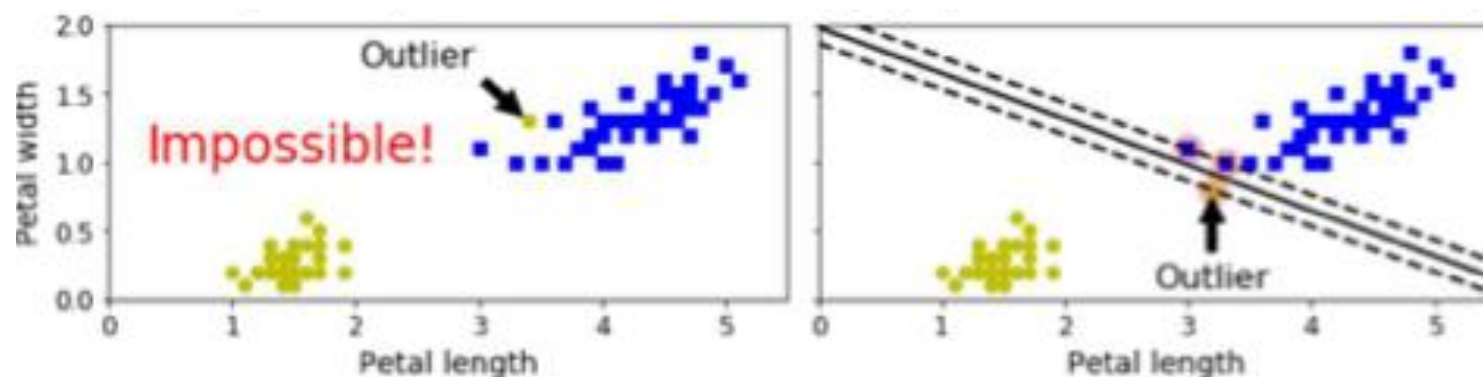
Sensibilité aux échelles des variables



- Les SVM sont sensibles aux différences d'échelles des variables.
- **A gauche** : L'échelle verticale est + grande que l'échelle horizontale c'est pour ça que le chemin le + large est proche de l'horizontale.
- **A droite** : après normalisation (par exemple en utilisant la fonction StandardScaler, la frontière de décision sur ce graphique parait bien meilleure.

Classification SVM à marge souple

- Lorsqu'on impose que toutes les observations soient en dehors du chemin et du bon côté, on applique une **classification rigide (hard margin)**.
- **Principaux inconvénients de la classification rigide** : elle ne fonctionne que si les données sont séparables linéairement + elle est sensible aux données aberrantes.
- Solution : **classification marge souple (soft margin)** trouver un bon équilibre entre conserver le chemin aussi large que possible et limiter les empiètements de marge (le nbr d'observations se trouvant à l'intérieur du chemin voire même du mauvais côté).



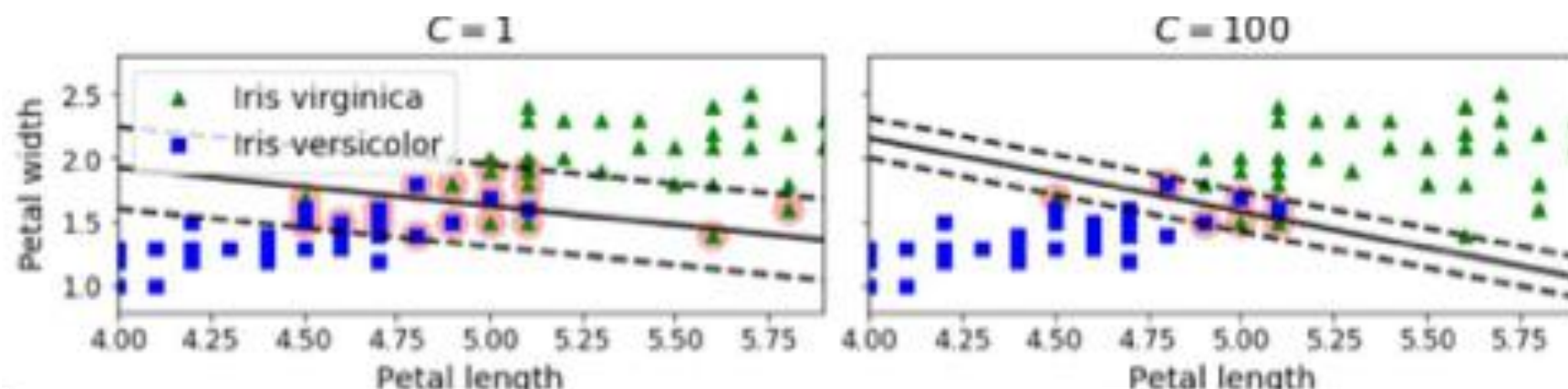
Ajout d'une donnée aberrante

À gauche : impossible de trouver une marge rigide

À droite : frontière devient très $\neq$, ça ne généralisera pas bien

Classification SVM à marge souple

- Lors de la création d'un modèle SVM dans scikit learn, nous pouvons spécifier un certain nombre d'hyperparamètres dont **l'hyperparamètre C**.
- Si C a une valeur trop basse => modèle de gauche.
- Si C a une valeur élevée => modèle de droite.
- Il est préférable d'avoir peu d'empiètement de marge. Ici le modèle de gauche présente beaucoup d'empiètements de marge mais généralisera probablement mieux.
- Si le modèle surajuste, on peut tenter de le régulariser en diminuant C.



Marge + large à gauche ou moins d'empiètements de marge à droite

Classification SVM à marge souple

Ce code charge le jeu de données Iris, normalize les variables puis entraine un modèle SVM linéaire via *LinearSVC* avec $C=1$ et la fonction coût charnière (*hinge loss*) dans le but de détecter Iris virginica.

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # longueur pétale et largeur pétale
y = (iris["target"] == 2).astype(np.float64) # 1 si Iris virginica, 0 sinon

svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=1, loss="hinge", random_state=42)),])
svm_clf.fit(X, y)
print(svm_clf.predict([[5.5, 1.7]])) #prediction sur une nouvelle donnée
```

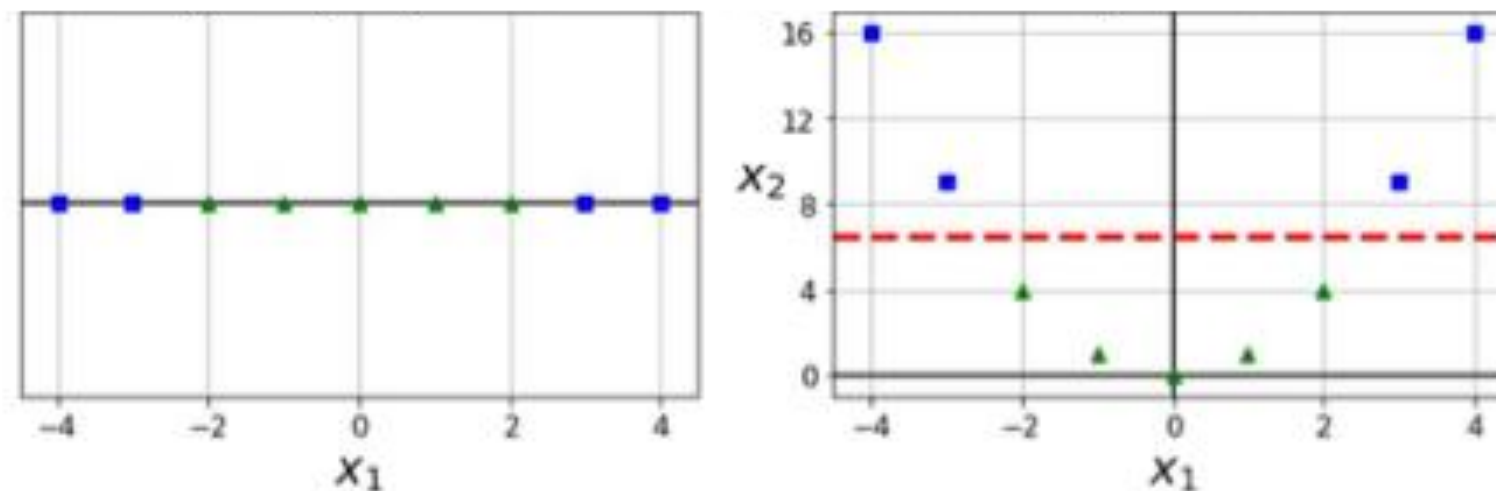
Affichage :
[1.]

Classification SVM à marge souple

- Au lieu de la classe *LinearSVC*, nous pourrions utiliser la classe *SVC* avec un noyau linéaire.
- A la création du modèle *SVC*, nous écrivons *SVC(kernel="linear" , C=1)*.
- Il est possible aussi d'utiliser la classe *SGDClassifier* sous la forme *SGDClassifier(loss="hinge", alpha=1/m*C)*, ce qui fait une descente de gradient stochastique normale pour entraîner un classifieur SVM linéaire.
- Ce classifieur *SGDClassifier* ne converge pas aussi rapidement que *LinearSVC*, mais ça peut être utile pour gérer des tâches de classification en ligne ou lorsqu'on a de grands jeux de données qui ne tiennent pas en mémoire.
- A noter que la classe *LinearSVC* régularise le terme constant, c'est pourquoi on doit d'abord centrer le jeu d'entraînement en soustrayant sa moyenne. Ce qui est fait automatiquement si on normalise les données avec *StandardScaler*.
- Il faut donner aussi spécifiquement à l'hyperparamètre *loss* la valeur *hinge* car c'est pas la valeur par défaut.
- Aussi pour améliorer les performances, il faut donner à l'hyperparamètre *dual* la valeur *false* sauf si il y a plus de variables que d'observations d'entraînement.

Classification SVM non-linéaire

- L'idée de base : ajouter d'avantage de variables, par exemple des variables polynomiales, dans certains cas le jeu résultant est linéairement séparable.
- Le jeu de données de cette figure n'est pas linéairement séparable (figure gauche) .
- Si on rajoute une 2ème variable $x_2 = x_1^2$, le jeu résultant devient linéairement séparable.

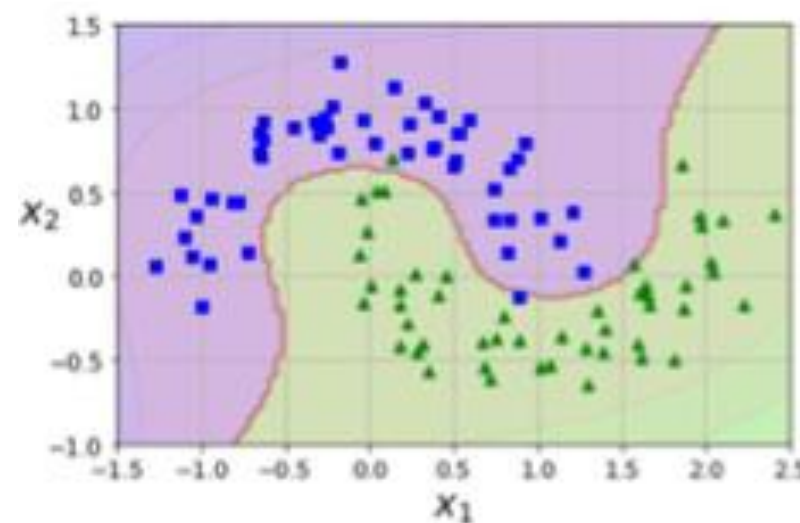


Marge + large à gauche ou moins d'empiètements de marge à droite

Classification SVM non-linéaire

- Pour l'implémentation, on peut créer un Pipeline composé d'un transformateur *PolynomialFeatures* suivi d'un *StandardScaler* et d'un *LinearSVC*.
- Testons le sur un nouveau jeu de données moons généré via *make_moons()*

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=3)),
    ("scaler", StandardScaler()),
    ("svm_clf", LinearSVC(C=10, loss="hinge", random_state=42))
])
polynomial_svm_clf.fit(X, y)
```



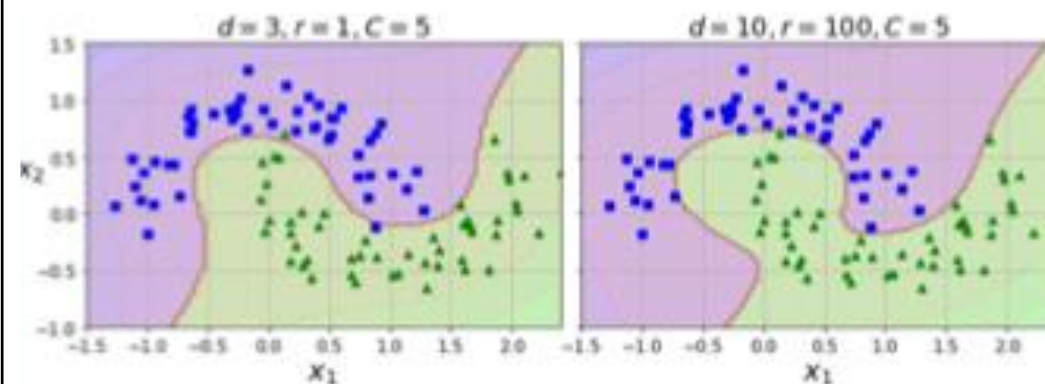
Classificateur SVM utilisant des données polynomiales

Classification SVM non-linéaire : noyau polynomial

- Cette méthode permet d'avoir les mêmes résultats que si on a plusieurs variables polynomiales sans avoir à les rajouter. L'astuce du noyau est implémentée via la classe SVC
- Si le modèle surajuste => réduire le degré polynomial. S'il sous-ajuste => augmenter le degré polynomial.
- $coef0$ contrôle l'influence relative sur le modèle des polynômes de degré élevé par rapport à ceux de degré faible.

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X, y)

poly100_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=10, coef0=100,
C=5))
])
poly100_kernel_svm_clf.fit(X, y)
```

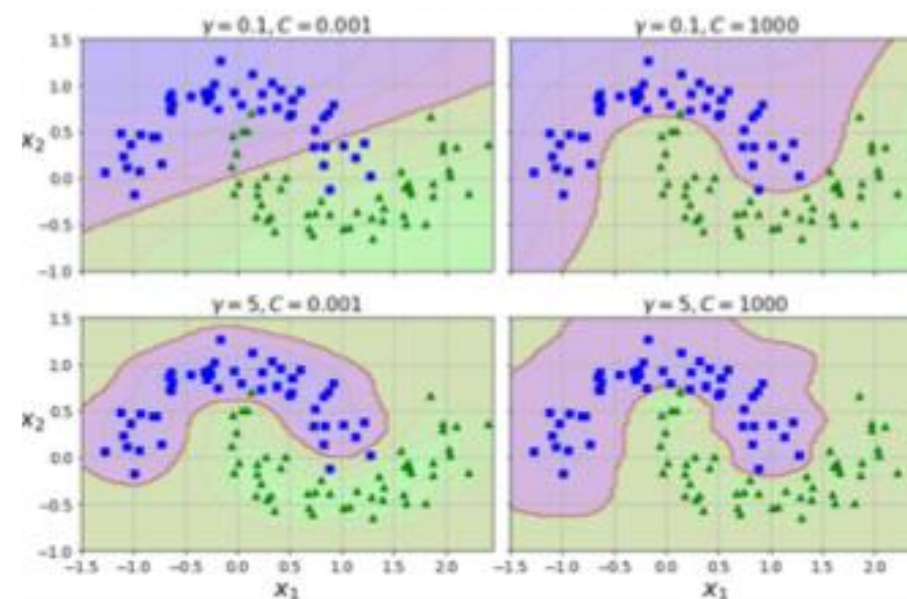


Classificateur SVM utilisant
avec noyau polynomial

Classification SVM non-linéaire : noyau radial gaussien

- Elle permet d'avoir les mêmes résultats que si on a ajouté plusieurs variables de similarité.
- **Accroître gamma** rend la courbe en **cloche plus étroite** et diminue l'intervalle d'influence de chaque observation donc la **frontière de décision devient plus irrégulière** se faufilant autour des $\neq$ observations.
- **Réduire gamma** rend la courbe en **cloche plus large** et augmente l'intervalle d'influence de chaque observation donc la **frontière de décision devient plus lisse**.

```
from sklearn.svm import SVC
gamma1 = 5
C1 = 0.001
rbf_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="rbf", gamma=gamma1, C=C1))
])
rbf_kernel_svm_clf.fit(X, y)
```



- Gamma fonctionne comme un hyperparamètre de régulation.
- Lorsque le modèle **surajuste** réduire le gamma.
- Lorsque le modèle **sous-ajuste** augmenter le gamma.

Mesures de performances : validation croisée

- Un bon moyen d'évaluer un modèle est d'effectuer une validation croisée.

Une validation croisée à K plis = partager le jeu d'entraînement en K blocs, effectuer des prédictions et les évaluer sur chaque bloc en utilisant un modèle entraîné sur le reste des blocs.

- Pour implémenter la validation croisée, on utilise `cross_val_score` comme suit :

```
from sklearn.model_selection import cross_val_score  
cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
```

Affichage : [0.96355, 0.93795, 0.95615]

Pour avoir le score global, on calcule la moyenne des scores donnés par cross-val-score.

Mesures de performances : matrice de confusion

- La matrice de confusion est un tableau qui compare les prédictions du modèle aux vraies valeurs afin de visualiser ses erreurs et ses bonnes classifications.

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specifitivity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TN + FP + FP + FN)}$

- Vrais positifs (VP/TP)** = des individus classés malades qui le sont vraiment.
- Vrais négatifs (VN/TN)** = individus classés sains qui le sont vraiment
- Faux positifs FP** = individus classés comme malades alors qu'ils sont sains
- Faux négatifs FN** = individus classés sains alors qu'ils sont malades

Mesures de performances : matrice de confusion

Exemple d'une classification binaire malade/sain

		Classes estimées par le classifieur	
		malade	sain
Classes réelles	malade	95 Vrais positifs	5 Faux négatifs
	sain	3 Faux positifs	97 Vrais négatifs

Exemple d'une classification Iris

de\vers	Setosa	Versicolor	Virginica	Total	% correct
Setosa	50	0	0	50	100%
Versicolor	0	47	3	50	94%
Virginica	0	1	49	50	98%
Total	50	48	52	150	97,33%

Mesures de performances : matrice de confusion

L'implémentation d'une matrice de confusion se fait via l'instruction *confusion_matrix* comme suit :

```
from sklearn.model_selection import cross_val_predict  
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)  
  
from sklearn.metrics import confusion_matrix  
confusion_matrix(y_train_5, y_train_pred)
```

Affichage : $\begin{bmatrix} 53057 & 1522 \\ 1325 & 4096 \end{bmatrix}$

```
y_train_perfect_predictions = y_train_5 # lorsqu'on a une prédiction parfaite  
confusion_matrix(y_train_5, y_train_perfect_predictions)
```

Affichage : $\begin{bmatrix} 54579 & 0 \\ 0 & 5421 \end{bmatrix}$

Mesures de performances : Précision, Rappel, F1

$$\textit{Précision} = \frac{VP}{VP + FP}$$

$$\textit{Rappel} = \frac{VP}{VP + FN}$$

$$F1 = \frac{2}{\frac{1}{\textit{Précision}} + \frac{1}{\textit{Rappel}}}$$

```
#classification binaire
from sklearn.metrics import precision_score, recall_score, f1_score
print(precision_score(y_train, y_train_pred))
print(recall_score(y_train, y_train_pred))
print(f1_score(y_train, y_train_pred))
```

```
#classification multi classes :
print("Classification report for classifier %s:\n%s\n" % (classifier, metrics.classification_report(y_test, predicted)))
disp = metrics.plot_confusion_matrix(classifier, X_test, y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix:\n%s" % disp.confusion_matrix)
plt.show()
```

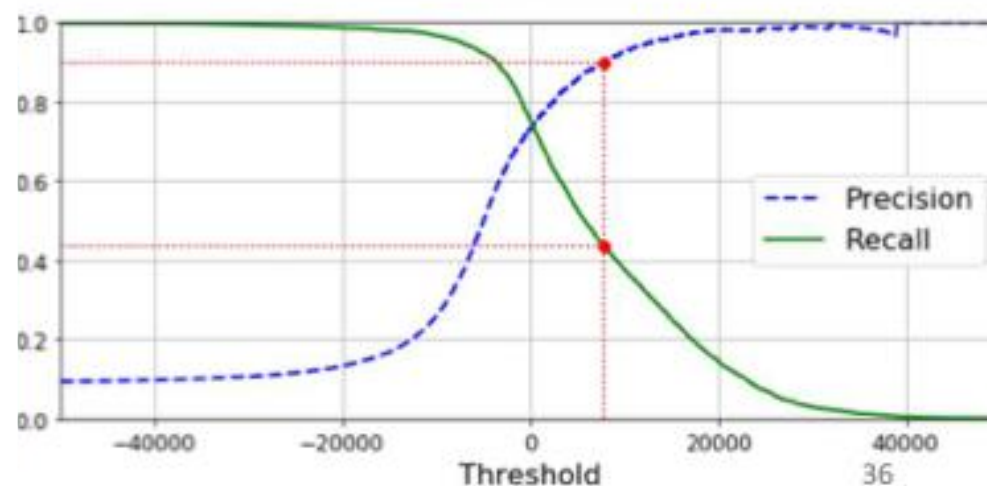
Mesures de performances : Compromis Précision, Rappel



Comment est ce que le classifieur SGDClassifier prend ses décisions ?

Pour chaque observation, il calcule un score basé sur **une fonction de décision**. Si $\text{score} > \text{seuil}$, il affecte l'observation à la classe positive, sinon il affecte à la classe négative

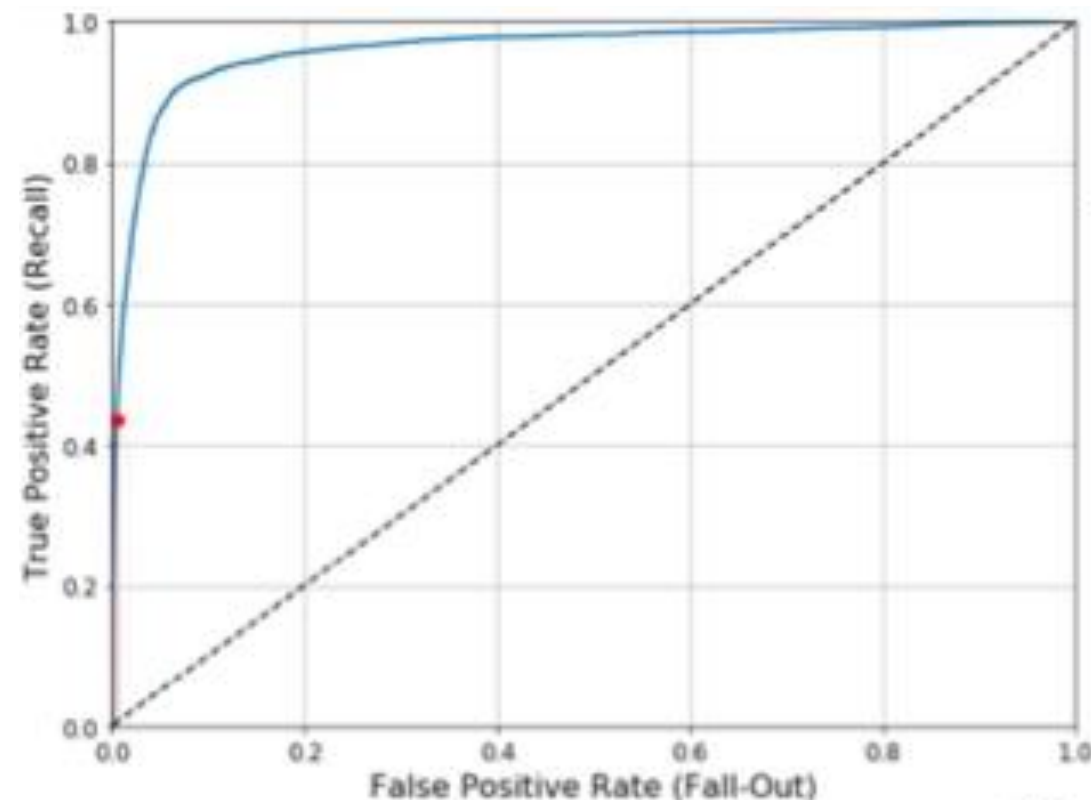
```
y_scores = cross_val_predict(sgd_clf, X_train, y_train, cv=3, method="decision_function")  
from sklearn.metrics import precision_recall_curve  
precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores)  
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)  
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
```



Mesures de performances : la courbe ROC

C'est une courbe qui trace le taux des vrais positifs en fonction du taux de faux positifs pour tous les seuils possibles.

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
plt.plot(fpr, tpr, linewidth=2, label=label)
plt.plot([0, 1], [0, 1], 'k--') # dashed diagonal
plt.axis([0, 1, 0, 1])
plt.xlabel('False Positive Rate (Fall-Out)', fontsize=16)
plt.ylabel('True Positive Rate (Recall)', fontsize=16)
plt.grid(True)
```



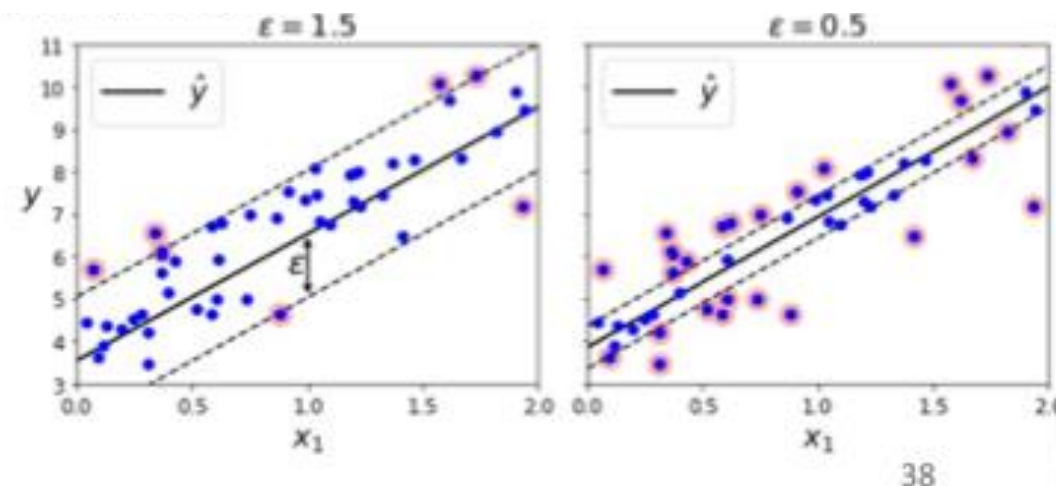
Régression SVM linéaire

- Au lieu d'essayer d'ajuster le chemin le + large possible entre 2 classes en limitant les empiètements de marge, là :

La régression SVM s'efforce d'ajuster autant d'observations que possible sur le chemin en limitant les empiètements.

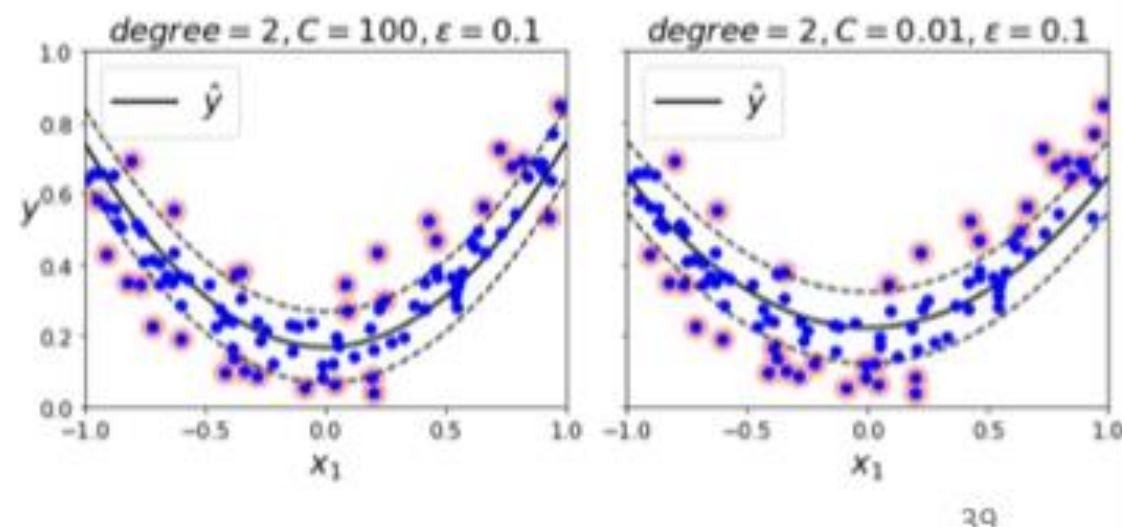
- La largeur du chemin est contrôlé par l'hyperparamètre ϵ
- Ajouter des observations à l'intérieur de la marge n'affecte pas le modèle.
- Le modèle est insensible à ϵ près.
- Le code suivant donne le résultat de la figure de gauche
- Les données doivent être normalisées et centrées.

```
from sklearn.svm import LinearSVR
svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```



Régression SVM non-linéaire

- On utilise le modèle SVM à noyau.
- Voici le résultats sur un jeu de d'entrainement quadratique aléatoire en utilisant un noyau polynomial de degre 2.
- A gauche on a une faible régularisation C élevée.
- A droite, on a une régularisation + importante (C + petit).
- La classe *SVR* est équivalente pour la régression à *SVC*.
- La classe *LinearSVR* est équivalente pour la régression à *LinearSVC*



```
from sklearn.svm import SVR
svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1, gamma="scale")
svm_poly_reg.fit(X, y)
```

K Nearest Neighbors (k-NN)

Principe de k-NN : Dis moi qui sont tes voisins, je te dirais qui tu es !

- C'est la méthode la plus simple d'apprentissage supervisé appelée « la méthode des k plus proches voisins : kppv».
- Pour effectuer une prédiction, l'algorithme k-NN ne va pas calculer un modèle prédictif à partir d'un ensemble d'entraînement comme c'est le cas pour la régression linéaire ou la régression polynomiale.
- Pour k-NN il n'existe pas de phase d'apprentissage au sens classique. On dit qu'il fait partie du **Lazy Learning**.
- Pour pouvoir effectuer une prédiction, l'algorithme k-NN se base sur le jeu de données pour produire un résultat.

Comment k-NN effectue une prédiction ?

- Pour faire une prédiction, cet algorithme se base sur le jeu de données en entier.
- Pour une observation, qui ne fait pas parti du jeu de données qu'on souhaite prédire, l'algorithme va **chercher les k instances du jeu de données les plus proches de notre observation**.
- Puis, pour ces k voisins, l'algorithme se basera sur leurs variables de sortie (output variable) y pour calculer la valeur de la variable y de l'observation qu'on souhaite prédire.
- Si k-NN est utilisé pour **la régression, c'est la moyenne (ou la médiane)** des variables y des k plus proches observations qui servira pour la prédiction.
- Si k-NN est utilisé pour **la classification, c'est la classe majoritaire** des variables y des k plus proches observations qui servira pour la prédiction.

L'algorithme k-NN

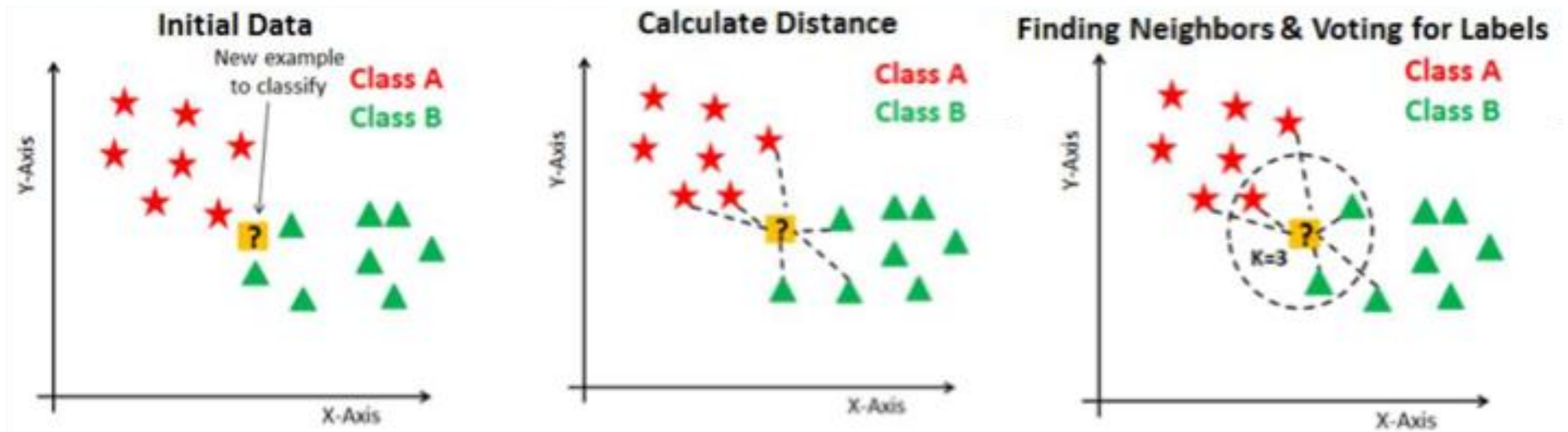
En entrée :

- Un ensemble de données D
- Une fonction de définition distance d
- Un nombre entier k

Pour une nouvelle observation X dont on veut prédire sa variable de sortie y Faire :

1. Calculer toutes les distances de cette observation X avec les autres observations du jeu de données D
2. Retenir les k observations du jeu de données D les proches de X en utilisation le fonction de calcul de distance d
3. Prendre les valeurs de y des k observations retenues :
 - i. Si on effectue une régression, calculer la moyenne (ou la médiane) de y retenues
 - ii. Si on effectue une classification , calculer le mode de y retenues
4. Retourner la valeur calculée dans l'étape 3 comme étant la valeur qui a été prédite par k-NN pour l'observation X .

L'algorithme k-NN



Calcul de distance dans k-NN

- La distance Euclidienne : $D(x, y) = \sqrt{\sum_{j=1}^n (x_j - y_j)^2}$
- La distance de Manhattan : $D(x, y) = \sum_{j=1}^n |x_j - y_j|$
- La distance de Hamming : $D(x, y) = \sum_{j=1}^n 1(x_j \neq y_j)$

String A

1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

String B

1	0	0	1	0	0	0	1
---	---	---	---	---	---	---	---

Compare:

=	=	≠	=	=	≠	=	=
---	---	---	---	---	---	---	---

La distance de Hamming mesure le nombre de positions où **deux chaînes de même longueur** ont des symboles différents. Elle sert de mesure de similarité pour les données binaires ou catégorielles. La simplicité de la métrique et l'efficacité de son calcul la rendent particulièrement utile lorsque l'on travaille avec des données binaires de haute dimension.

Calcul de distance dans k-NN

On choisit la fonction de distance en fonction des types de données qu'on manipule.

Distance euclidienne

Pour des données numériques continues (ex : poids, salaires, taille, ...)

Distance de Manhattan

Pour la mesure en suivant les axes (utile lorsque les déplacements se font par dimensions indépendantes)

Distance de Hamming

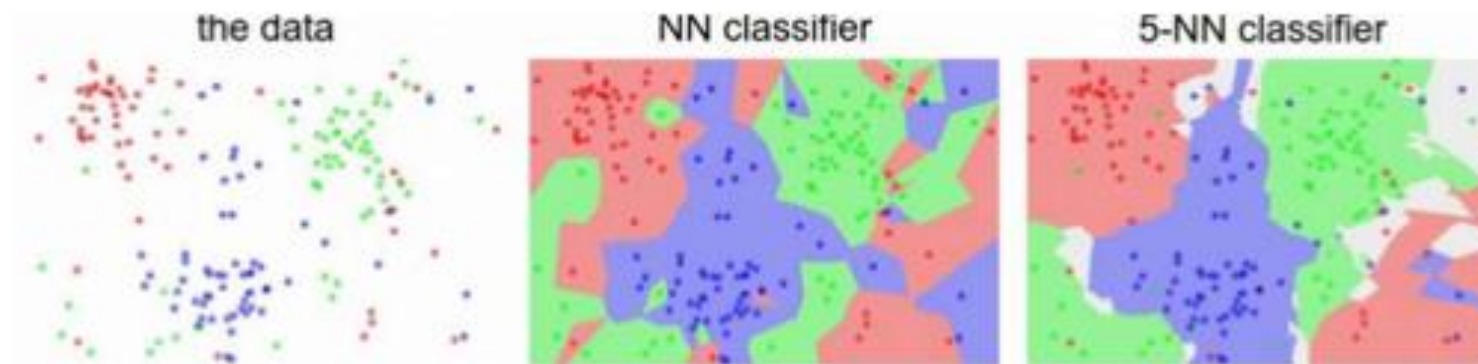
Pour des données discrètes ou catégorielles (ex : chaînes de caractères, variables binaires)

Le choix de la distance influence directement les performances du modèle.

Choix de la valeur de k dans k -NN

On choisit le nombre de voisins (k) en fonction du jeu de données qu'on utilise.

- Moins on utilise de voisins (un nombre k petit) plus on est sujet au sous-apprentissage (underfitting).
- Plus on utilise de voisins (un nombre k grand) plus on est fiable dans notre prédiction.



Si on utilise k nombre de voisins avec $k=N$ et N étant le nombre d'observations, on risque d'avoir du sur-apprentissage (overfitting).

Classification avec k-NN

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Charger et séparer les données
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Créer et entraîner le modèle
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Prédire et évaluer
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Régression avec k-NN

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.neighbors import KNeighborsRegressor

X, y = make_regression(n_samples=100, n_features=1, noise=20, random_state=42)
X = np.sort(X, axis=0) # Trier les données pour un affichage propre

model = KNeighborsRegressor(n_neighbors=5)
model.fit(X, y)

X_test = np.linspace(X.min(), X.max(), 100).reshape(-1, 1) # Points pour la prédiction
y_pred = model.predict(X_test)

plt.scatter(X, y)      # données réelles
plt.plot(X_test, y_pred) # prédiction k-NN
plt.title("Régression avec k-NN")
plt.xlabel("X")
plt.ylabel("y")
plt.show()
```

Limitations de k-NN

- k-NN est un algorithme assez simple à appréhender puisqu'il n'a pas besoin de modèle pour pouvoir effectuer une prédiction.
- Le contre coût est qu'il doit garder en mémoire l'ensemble des observations pour pouvoir effectuer sa prédiction. Ainsi il faut faire attention à la taille du jeu d'entraînement.
- Également, le choix de la méthode de calcul de la distance ainsi que le nombre de voisins k peut ne pas être évident.
- Il faut essayer plusieurs combinaisons de l'algorithme pour avoir un résultat satisfaisant.

III. Modélisation supervisée et non-supervisée en Machine Learning



Savoir choisir et mettre en œuvre des modèles supervisés et non supervisés adaptés aux données

3.1

Régression : prédire des valeurs continues

3.2

Classification : assigner des classes

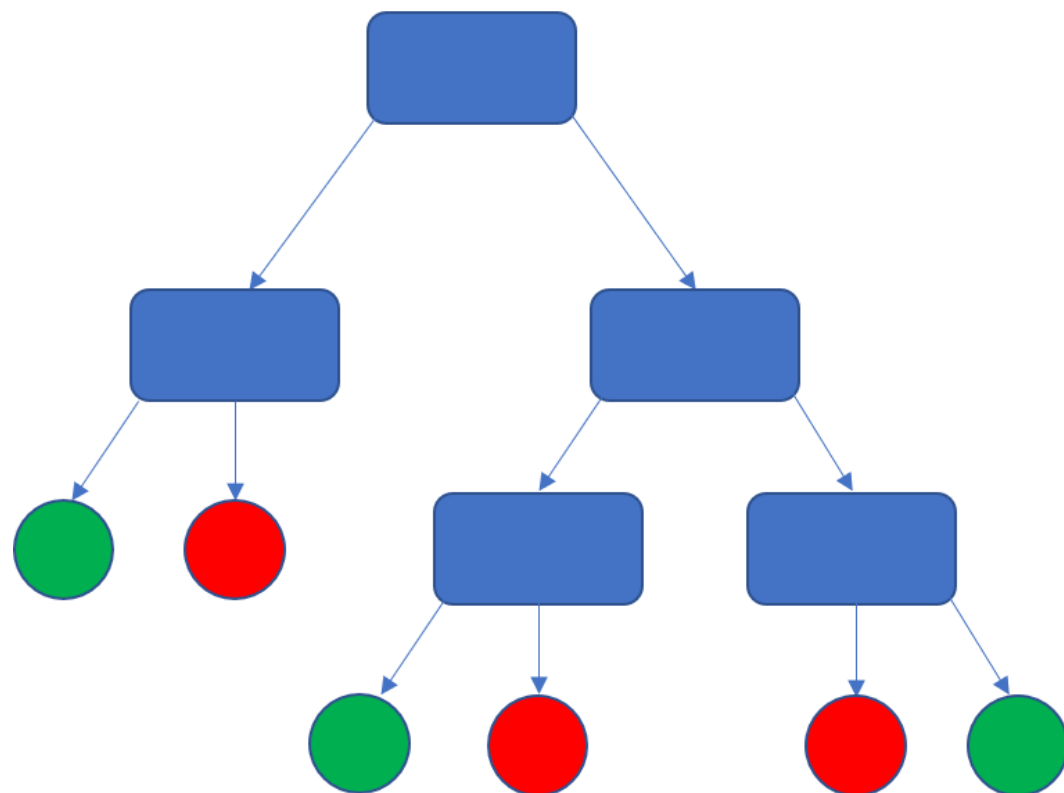
3.3

Arbres de décision : apprendre des règles

3.4

Clustering : découvrir des groupes

Objectif



Créer un modèle qui prédit les valeurs de la variable cible, en se basant sur un ensemble de séquences de règles de décision déduites à partir des données d'apprentissage.

Arbres de décision

L'arbre approxime la cible par une succession de règles "if-then-else".

- Ils s'appliquent aussi bien à des données catégorielles qu'aux données numériques et le coût d'utilisation des arbres est logarithmique.
- Plus l'arbre généré est complexe, mieux le modèle « explique » les données d'apprentissage mais plus le risque de sur-apprentissage (overfitting) est élevé.
- Ils sont simples à comprendre et à visualiser et nécessitent peu de préparation des données.
- Ils sont capables de traiter des problèmes multi-classes.



Modèle en boîte blanche : le résultat est facile à conceptualiser et à visualiser.

Arbres de décision

L'arbre approxime la cible par une succession de règles "if-then-else".

- **Sur-apprentissage** : parfois les arbres générés sont trop complexes et généralisent mal. Choisir des bonnes valeurs pour les paramètres profondeur maximale (*max_depth*) et nombre minimal d'exemples par feuille (*min_samples_leaf*) permet d'éviter ce problème.
- Il peut arriver que **les arbres générés ne soient pas équilibrés** (ce qui implique que le temps de parcours n'est plus logarithmique). Il est donc recommandé d'ajuster la base de données avant la construction, pour éviter qu'une classe domine largement les autres (en termes de nombre d'exemples d'apprentissage).

Installation de graphviz

1. Installer Graphviz (logiciel système)

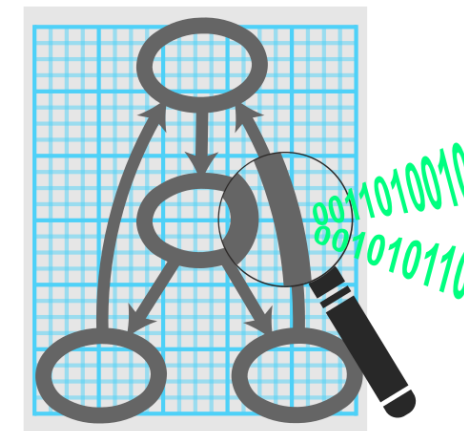
- Aller sur le site officiel : <https://graphviz.org/download/>
- Télécharger la version correspondant à votre système (Windows / Mac / Linux) puis installer en suivant les étapes
- ⚠ Important : cocher l'option pour **ajouter Graphviz au PATH** (ou le faire manuellement)

2. Installer les bibliothèques Python

- Ouvrir un terminal (Invite de commandes / Terminal / PowerShell) puis exécuter :
- `pip install graphviz`
- `pip install python-graphviz`

3. Vérification (optionnel)

- Dans Python, tester :
- `import graphviz`



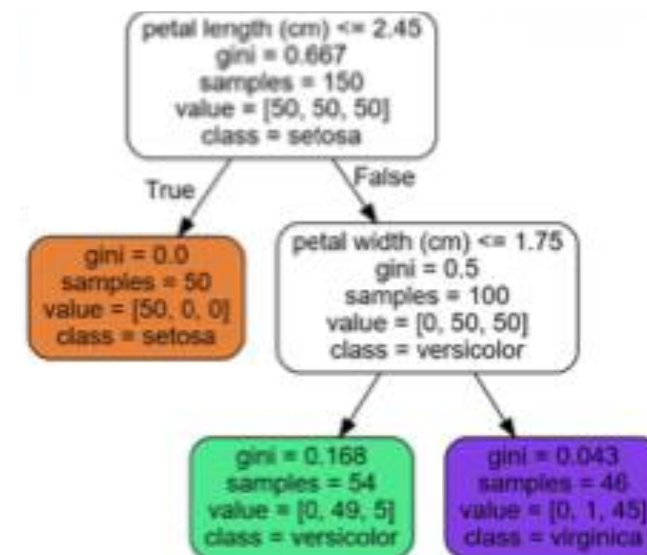
Entraîner et visualiser un arbre de décision

- Construisons un arbre de décision et voyons comment il réalise ses prédictions :

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X = iris.data[:, 2:] # longueur et largeur des pétales
y = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X, y)
```

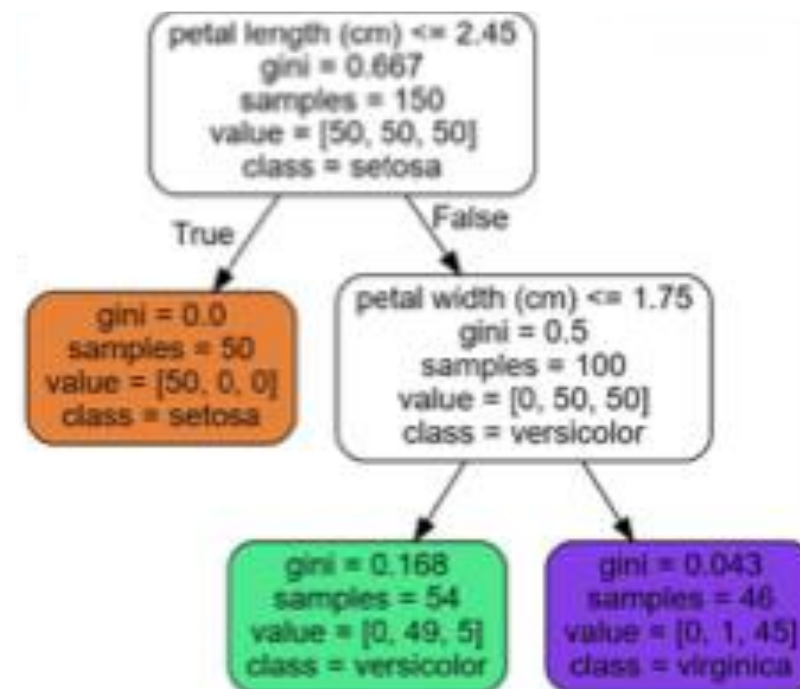
- On visualise l'arbre via `export_graphviz()` qui fournit en sortie un fichier de définition graphique `iris_tree.dot` :

```
import os
from graphviz import Source
from sklearn.tree import export_graphviz
export_graphviz(tree_clf, out_file=os.path.join("iris_tree.dot"), feature_names=iris.feature_names[2:],
                class_names=iris.target_names, rounded=True, filled=True)
Source.from_file(os.path.join("iris_tree.dot"))
```



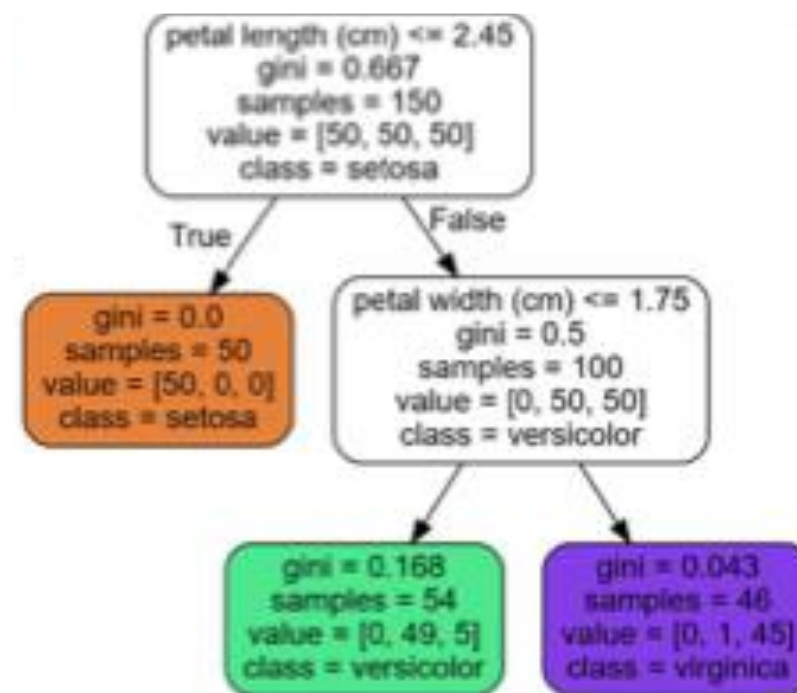
Entraîner des prédictions

- Supposons qu'on a une fleur d'iris à classer:
 - On commence par le nœud racine. Est-ce que la longueur de pétale ≤ 2.45 cm ?
 - Si c'est le cas on va vers le nœud inférieur gauche. C'est une feuille donc aucune autre question. On regarde la classe, ça sera une fleur d'iris de type **setosa**.
 - Si la longueur pétale > 2.45 cm , on va vers le nœud inférieur droit. On pose une autre question : est ce que la largeur pétale ≤ 1.75 cm ? Si c'est le cas la fleur est une **versicolor** sinon c'est une **virginica**.



Entraîner des prédictions

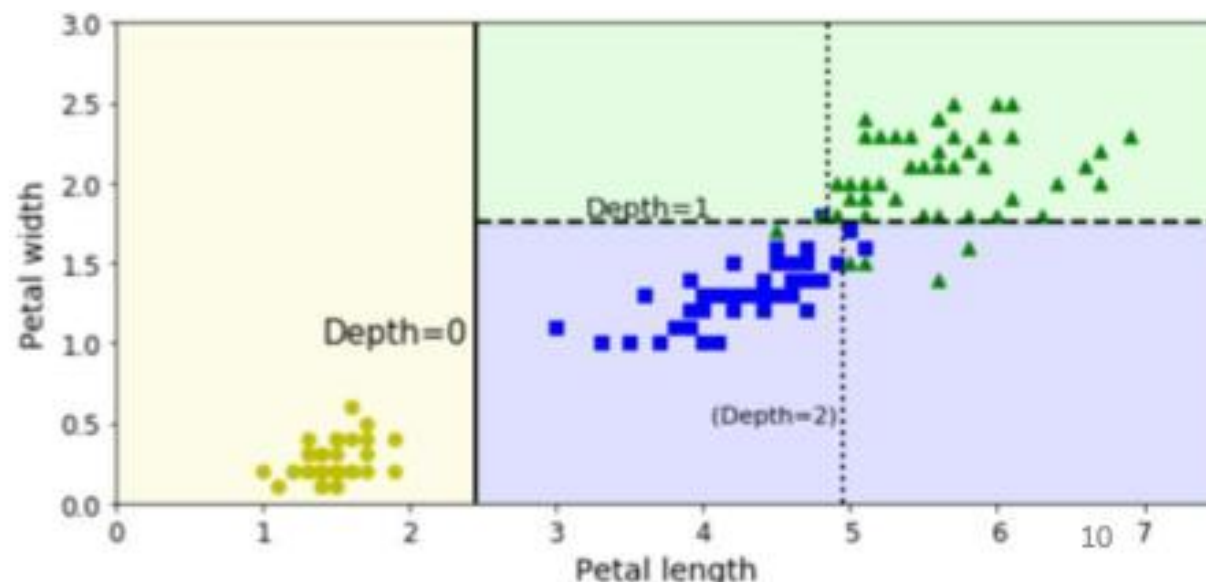
- **L'attribut samples** compte le nombre d'observations d'entraînement passées par ce nœud.
- On peut voir que 100 observations d'entraînement sont passées par le nœud de profondeur 1 à droite parce que leur longueur de pétale est $> 2.45\text{cm}$.
- Parmi celles-ci 54 ont une largeur de pétale $> 1.75\text{ cm}$ (nœud de profondeur 2 à gauche).
- **L'attribut value** d'un nœud indique combien d'observations d'entraînement de chaque classe sont passées par là.
- Le nœud en bas à droite a été atteint par 0 setosa, 1 iris versicolor et 45 virginica.
- **L'attribut gini** d'un nœud mesure son impurité.
- Un nœud est pur ($\text{gini}=0$) si toutes les observations d'entraînement qui aboutissent sont de la même classe.
- Le nœud gauche de profondeur 1 ne reçoit que des observations d'entraînement de setosa, il est pur, $\text{gini}=0$.



Entraîner des prédictions

Les frontières de décision de l'arbre étudié :

- La ligne verticale (trait plein) représente la frontière de décision du nœud racine (longueur pétal 2.45cm)
- La zone gauche est une zone pure setosa, elle ne peut être partagée.
- La zone de droite est impure, le nœud de droite de profondeur 1 établit la séparation à la largeur pétale 1.75 (ligne tiret).
- Etant donné que la profondeur maximale $max_depth = 2$, l'arbre s'arrête là.
- Si on avait donné à $max_depth = 3$ alors les deux nœuds de profondeur 2 auraient eu chacun une autre frontière de décision (lignes pointillées).

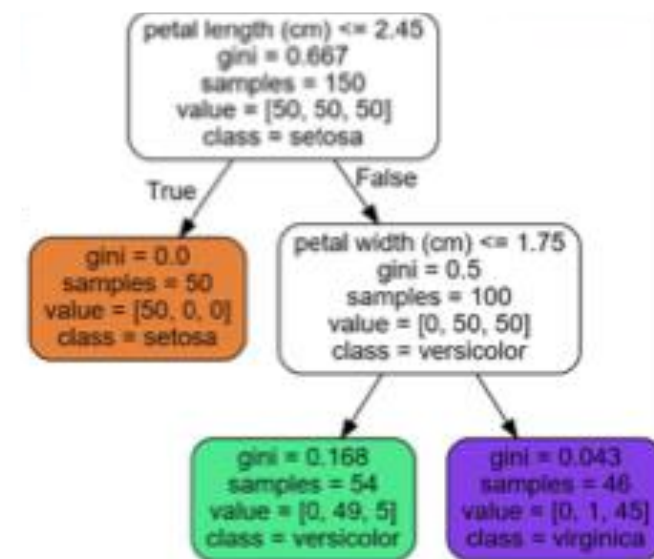


Estimation des probabilités par classe

- Un arbre peut estimer aussi la probabilité qu'une observation appartienne à une classe donnée k .
- Supposons qu'on ait trouvé une iris de longueur de pétale 5cm et de largeur 1.5cm.
- Le nœud terminal est le nœud gauche de profondeur 2. L'arbre renverra les probas 0% setosa (0/54), 90.7% versicolor (49/54) et 9.3% virginica (5/54). La prédiction donne la plus importante la classe versicolor.

```
tree_clf.predict_proba([[5, 1.5]])  
Affichage : array([[0. , 0.90740741, 0.09259259]])
```

```
tree_clf.predict([[5, 1.5]])  
Affichage: array([1])
```



Algorithme d'entrainement CART



Comment « faire pousser » les arbres ?

- L'algo sépare le jeu d'entrainement en 2 sous ensembles en utilisant une seule caractéristique k et un seuil t_k (longueur pétale ≤ 2.45 cm) .
- Comment choisit-il k et t_k ? Il recherche la paire (k, t_k) qui produit les sous ensembles les plus purs (pondérés par leur taille).
- La fonction de coût que l'algorithme essaie de minimiser est :

$$J(k, t_k) = \frac{m_{gauche}}{m} G_{gauche} + \frac{m_{droite}}{m} G_{droite}$$

$G_{gauche/droite}$ mesure l'impureté du sous-ensemble de gauche/droite

$m_{gauche/droite}$ est le nombre d'observations du sous-ensemble de gauche/droite

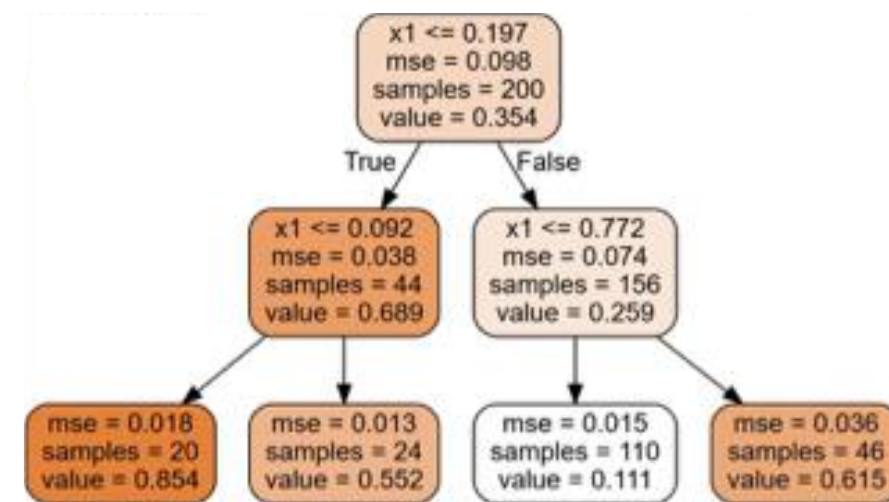
- Une fois le jeu partagé en 2, l'algo effectue la même logique aux sous ensembles afin de les partager et ainsi de suite.
- Cette récursivité ne s'interrompt que lorsque la profondeur maximale est atteinte ou qu'il n'existe plus de partage réduisant l'impureté.

Régression par arbre de décision

- Pour effectuer une régression, on peut utiliser *DecisionTreeRegressor* de scikit learn. Pour illustrer cela, l'algorithme suivant effectue une régression sur des données quadratiques comportant du bruit

```
np.random.seed(42)
m=200
X = np.random.rand(m, 1)
y=4*(X-0.5)**2
y = y + np.random.randn(m, 1) / 10

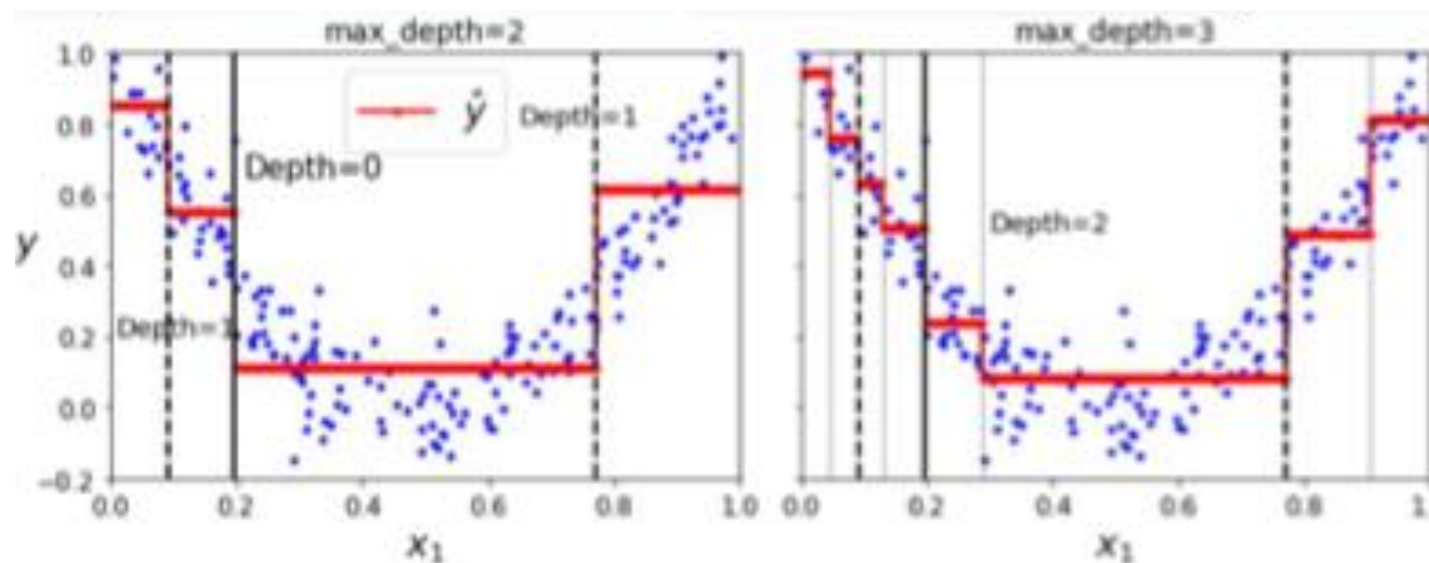
from sklearn.tree import DecisionTreeRegressor
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X, y)
```



- La différence avec la classification c'est au lieu de prédire une classe, il **prédit une valeur**.
- On veut une prédiction pour une nouvelle observation $x_1=0.6$. On traverse l'arbre à partir de la racine et on aboutit au nœud terminal prédisant $value=0.1106$ (valeur de 110 observations d'entraînement $MSE=0.0151$ sur 110 observations).

Régression par arbre de décision

- La valeur prédite pour chaque région est toujours la moyenne des valeurs cibles des observations dans cette région.



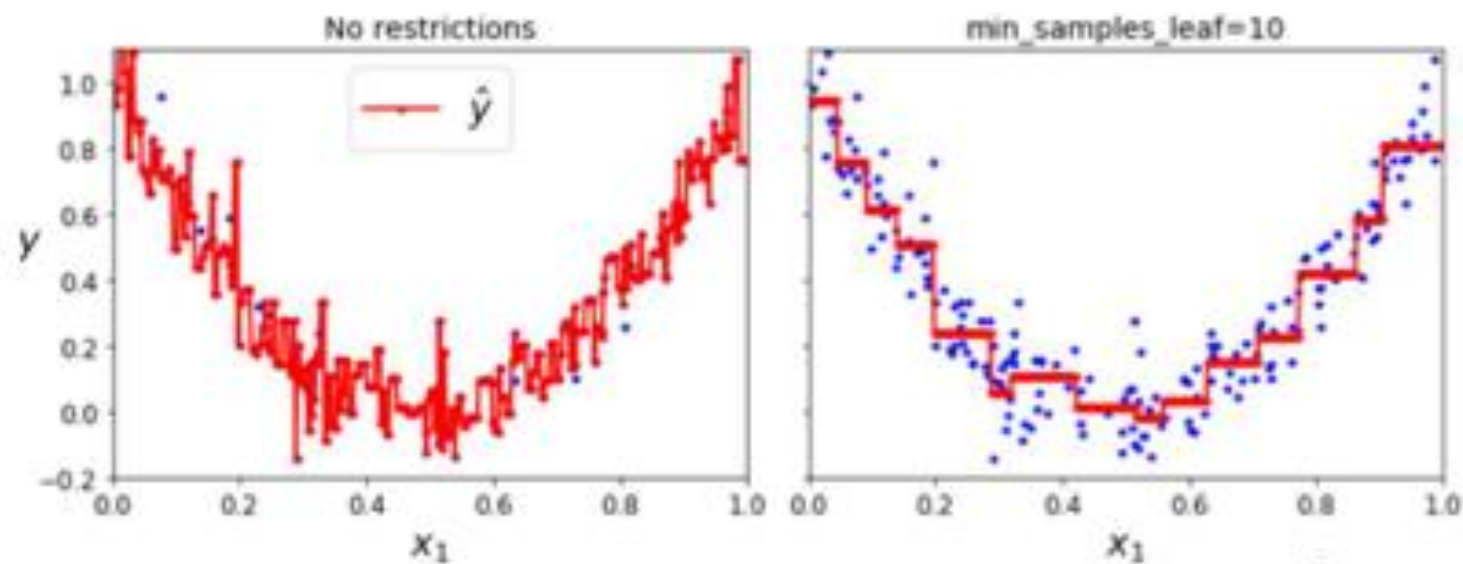
- L'algorithme CART fonctionne comme pour la classification sauf qu'au lieu de partager le jeu d'entraînement pour Minimiser l'impureté là il essaie de minimiser MSE.

$$J(k, t_k) = \frac{m_{gauche}}{m} MSE_{gauche} + \frac{m_{droite}}{m} MSE_{droite}$$

où $MSE_{noeud} = \sum_{i \in noeud} (\hat{y}_{noeud} - y^{(i)})^2$, avec $\hat{y}_{noeud} = \frac{1}{m_{noeud}} \sum_{i \in noeud} y^{(i)}$

Régression par arbre de décision

- Tout comme pour la classification, les arbres de décision ont tendance à surajuster lors de la régression.
- Sans régulation (c'est-à-dire en utilisant les paramètres par défaut), il se peut qu'on se retrouve dans le cas de la figure gauche : surajustement des données d'apprentissage.
- Pour y remédier on peut rajouter $min_samples_leaf = 10$ et on obtient la figure de droite.



III. Modélisation supervisée et non-supervisée en Machine Learning



Savoir choisir et mettre en œuvre des modèles supervisés et non supervisés adaptés aux données

3.1

Régression : prédire des valeurs continues

3.2

Classification : assigner des classes

3.3

Arbres de décision : apprendre des règles

3.4

Clustering : découvrir des groupes

Apprentissage non supervisé

Rappel : L'apprentissage non supervisé = un apprentissage sans étiquettes.



Pourquoi est ce qu'on n'a pas d'étiquettes ?

- Pas le temps ni l'argent
- Pas de spécialiste pour étiqueter
- Données impossible à étiqueter
- Évolution dynamique des structures
- Trop de catégories peu pertinentes
- L'objectif est de découvrir la structure des données, les motifs
- On ne sait pas ce qu'on cherche
- ...

Principe de base du Clustering

Clustering = regroupement (en français)

On n'essaie pas d'apprendre une relation de corrélation entre un ensemble d'entrées X d'une observation et une valeur de sortie à prédire Y , comme c'est le cas pour l'apprentissage supervisé.



Principe du clustering :

- Regrouper tout ce qui se ressemble.
- Eloigner tout ce qui est vraiment différent.
- Un cluster: un regroupement de données.

Clustering

- L'apprentissage non supervisé est très subjectif.
- L'objectif est induit par la formulation du problème.
- Ensemble très varié de techniques qui vise à trouver des sous-ensembles cohérents des données.
- Tout ce qui ressemble s'assemble $\Rightarrow$ définir une similarité entre exemples.
- Domaine-spécifique, pas de règle générale.
 - **Clustering hard** : un exemple n'appartient qu'à un groupe.
 - **Clustering soft** : probabilité d'appurtenance.



K-means : distance de similarité



- Permet de regrouper en K “clusters” distincts les observations d’un ensemble de données. Ainsi, les données similaires se retrouveront dans un même cluster.
- Une observation ne peut se retrouver que dans un cluster à la fois (**exclusivité d’appartenance**).
- Pour pouvoir regrouper un jeu de données en K cluster distincts, l’algorithme K-means a besoin d’un moyen de comparer le degré de similarité entre les différentes observations.
- Ainsi, deux données qui se ressemblent, auront une distance de dissimilarité réduite, alors que deux données différentes auront une distance de séparation plus grande.

K-means : choisir le nombre de clusters

- Choisir un nombre de cluster K n'est pas forcément intuitif. Spécialement quand le jeu de données est grand et qu'on n'ait pas un a priori ou des hypothèses sur les données.
- Un nombre **K grand** peut conduire à un **partitionnement trop fragmenté** des données. Ce qui empêchera de découvrir des patterns intéressants dans les données.
- Par contre, un nombre de clusters **trop petit**, conduira à avoir, potentiellement, **des cluster trop généralistes** contenant beaucoup de données. Dans ce cas, on n'aura pas de patterns "fins" à découvrir.
- Pour un même jeu de données, il n'existe pas un unique clustering possible. La difficulté résidera donc à choisir un nombre de cluster K qui permettra de mettre en lumière des patterns intéressants entre les données.
- **Il n'existe pas de procédé automatisé** pour trouver le bon nombre K de clusters.

K-means : choisir le nombre de clusters

- La méthode la plus usuelle : lancer K-Means avec différentes valeurs de K et de calculer la variance des différents clusters.
- La variance est la somme des distances entre chaque centroïde d'un cluster et les différentes observations incluses dans le même cluster.
- On cherche à trouver un K de telle sorte que les clusters retenus minimisent la distance entre leurs centres (centroïdes) et les observations dans le même cluster. On parle de minimisation de la distance intra-classe.
- La variance des clusters se calcule comme suit :

$$V = \sum_j \sum_{x_i \rightarrow c_j} D(c_j, x_i)^2$$

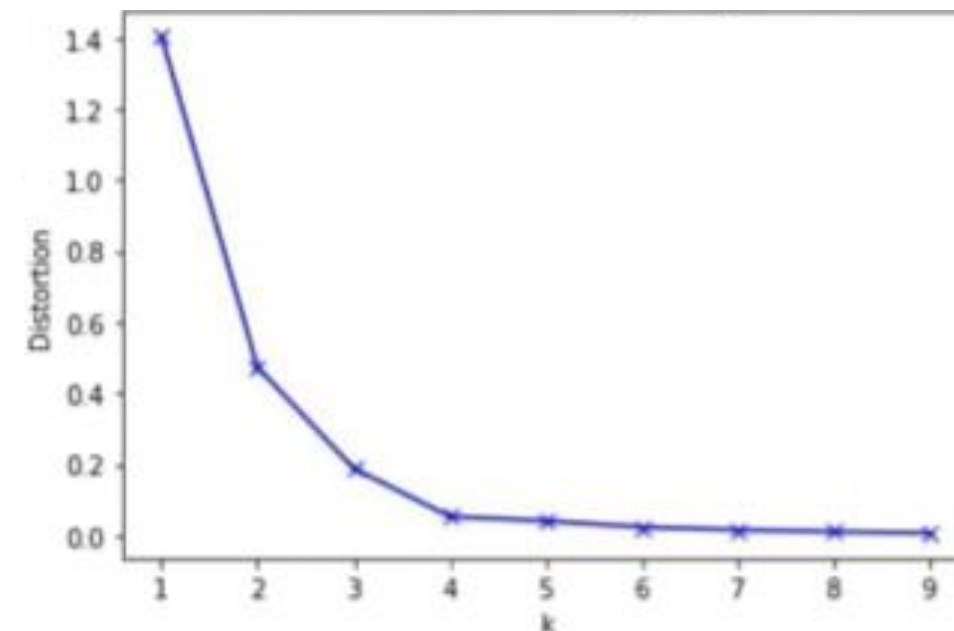
c_j : le centre du cluster (le centroïde)

x_i : la i ème observation dans le cluster ayant pour centroïde c_j

$D(c_j, x_i)$: la distance (euclidienne ou autre) entre le centre du cluster et le point x_i

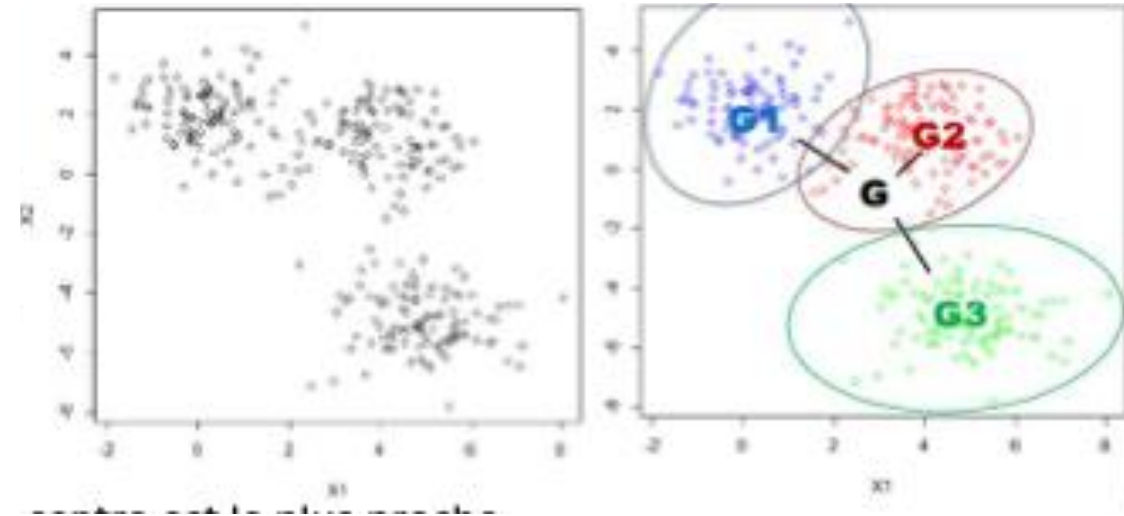
K-means : la méthode Elbow

- Graphique : nombres de clusters K en fonction de la variance.
- La forme d'un bras où le point le plus haut représente l'épaule et le point où K vaut 9 représente l'autre extrémité : la main.
- Le **nombre optimal de clusters K** est le point représentant le **coude**. Ici le coude peut être représenté par K valant 2 ou 3.



K-means : algorithme

- **Entrée** : X (entrées ou les observations), K (nombre de clusters)
 - Initialiser K centres de clusters G_k
 - REPETER
 - **Allocation**. Affecter chaque individu à la classe dont le centre est le plus proche
 - **Représentation**. Recalculer les centres de clusters à partir des individus rattachés
- JUSQU'À Convergence (Nombre d'itérations fixé ; Ou aucun individu ne change de classe; Ou encore lorsque l'inertie intra- classe ne diminue plus ; Ou lorsque les G_k sont stables)
- **Sortie** : Une partition des individus caractérisée par les K centres de clusters G_k



K-means via Scikit Learn

- Comment utiliser Kmeans via scikit-learn :

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=5, random_state=42)

k = 5
kmeans = KMeans(n_clusters=k, random_state=42)
y_pred = kmeans.fit_predict(X)

print("Labels des clusters :\n", y_pred)
print("\nCentres des clusters :\n", kmeans.cluster_centers_)

X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
print("\nPrédictions pour nouvelles données :")
print(kmeans.predict(X_new))
```

```
# Visualisation
plt.scatter(X[:, 0], X[:, 1], c=y_pred)
plt.scatter(kmeans.cluster_centers_[0],
            kmeans.cluster_centers_[1],
            marker='X', s=200)

plt.title("K-means clustering")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
```


Applications Kmeans

- Utilisation pour la segmentation d'images et leur compression
- La segmentation de la clientèle en fonction d'un certain critère (démographique, habitude d'achat etc....)
- Utilisation du clustering en Data Mining lors de l'exploration de données pour déceler des individus similaires. Généralement, une fois ces populations détectées, d'autres techniques peuvent être employées en fonction du besoin.
- Clustering de documents (regroupement de documents en fonction de leurs contenus. Pensez à comment Google Actualités regroupe des documents par thématiques.)



Utilisation de Kmeans pour la segmentation d'images

Segmentation de couleurs: affecter des pixels au même segment s'ils ont des couleurs similaires



Utilisation de Kmeans pour la segmentation d'images

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.image import imread
from sklearn.cluster import KMeans
from urllib.request import urlopen

url = "https://upload.wikimedia.org/wikipedia/commons/4/40/Sunflower_sky_backdrop.jpg"
image = imread(urlopen(url))

if image.dtype != np.float32 and image.dtype != np.float64:
    image = image / 255.0
if image.shape[-1] == 4: # Si RGBA → RGB
    image = image[..., :3]
print("Shape image :", image.shape)

X = image.reshape(-1, 3) # Transformer en liste de pixels

k = 8
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X)

segmented_img = kmeans.cluster_centers_[labels]
segmented_img = segmented_img.reshape(image.shape)
```

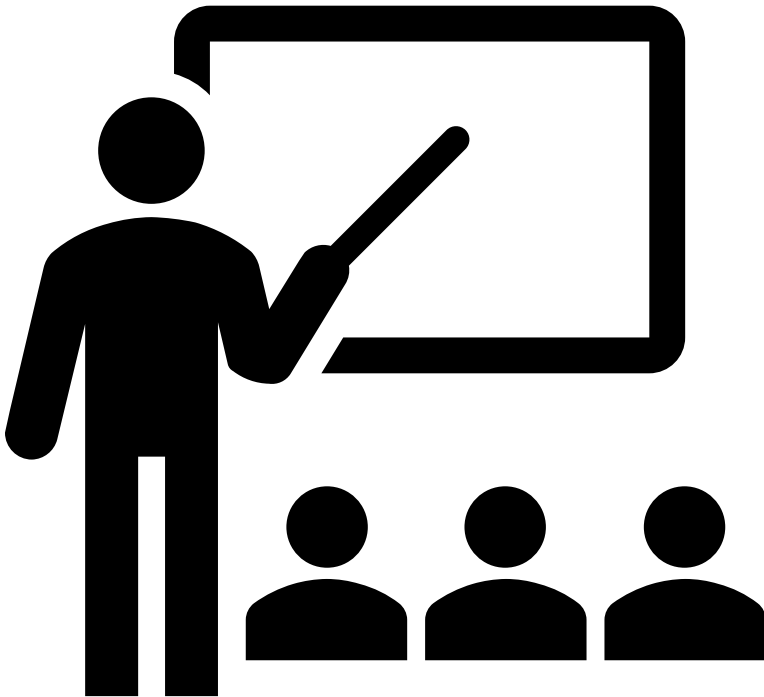
```
# 5) Affichage
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Image originale")
plt.axis("off")

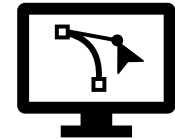
plt.subplot(1, 2, 2)
plt.imshow(segmented_img)
plt.title(f"Image segmentée (k={k})")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Contenu du cours



Fondamentaux du
Machine & Deep
Learning



Extraction, préparation,
visualisation et
communication des
données



Modélisation
supervisée et non-
supervisée en Machine
Learning



Modélisation
supervisée et avancée
en Deep Learning